INPT

Institut National
des Postes et
Télécommunications
Rabat

RAPPORT TECHNIQUE · PROJET SOA

Gestion intelligente des stages et des offres de recrutement

Trois services SOAP, trois services REST, un processus BPMN orchestré par Bonita et une interface pour les étudiants, les entreprises et le responsable des stages.

RÉALISÉ PAR

Hiba Elamdaoui
Aya Zahri
Hibatollah Ait Ahmed

ENCADRÉ PAR

Pr. [NOM À COMPLÉTER]

Filière AMOA
Module SOA
Année universitaire
2026-2027

Remerciements

Nous remercions Pr. [NOM À COMPLÉTER], qui a encadré ce projet dans le cadre du module SOA, pour un sujet qui nous a fait traverser toute la chaîne d'une architecture orientée services : contrats, sécurité des échanges, orchestration et tests.

Nos remerciements vont aussi à l'équipe pédagogique de la filière AMOA de l'INPT, dont les enseignements de modélisation des processus et d'analyse des besoins ont directement servi à la conception du processus de candidature.

Enfin, merci aux communautés qui maintiennent les outils libres utilisés ici : Eclipse Metro (JAX-WS), Eclipse Jersey (JAX-RS), MySQL, Mailpit, Bonita Community, SoapUI Open Source, Newman et bpmn-js.

Résumé

Ce rapport présente une plateforme de gestion des stages réalisée selon une architecture orientée services. Six services indépendants, chacun avec sa propre base de données, couvrent les étudiants, les entreprises partenaires et les offres de stage (services SOAP décrits par des contrats WSDL), ainsi que les candidatures, les notifications et le suivi des stages (services REST décrits par des contrats OpenAPI).

Le traitement d'une candidature, du dépôt à la décision de l'entreprise puis à la création du dossier de stage, est modélisé par un processus BPMN exécuté dans Bonita, qui appelle les six services. Onze règles de gestion, numérotées de R1 à R11, encadrent ce traitement : éligibilité de l'étudiant, validité de l'offre, date limite, places, doublons, nombre de candidatures actives, compatibilité du profil, annulation, transitions de statut, acceptation et demande de complément.

Les échanges SOAP sont authentifiés par WS-Security, les appels REST par une clé d'API par client, et les utilisateurs par un portail avec trois rôles. Les pannes de service sont traitées par le processus comme par le portail, et un identifiant de corrélation permet de suivre une candidature dans les journaux de tous les services.

Le projet est vérifié par 356 tests JUnit (dont 44 sur la vraie base de données), 546 assertions SoapUI, 226 assertions Postman et un test de bout en bout qui rejoue chaque scénario par le portail et contrôle les courriels reçus. Une interface web, proposée en bonus, permet aux étudiants de candidater, aux entreprises de décider et au responsable de suivre les stages, avec l'avancement de chaque candidature affiché sur le diagramme BPMN.

Mots-clés : architecture orientée services, SOAP, WSDL, JAX-WS, REST, JAX-RS, OpenAPI, BPMN, Bonita, orchestration, WS-Security, MySQL.

Table des matières

Remerciements i

Résumé ii

1 Introduction et contexte 1

1.1 Contexte . . . 1
1.2 Objectifs . . . 1
1.3 Choix techniques . . . 1
1.4 Organisation du rapport . . . 2

2 Analyse des besoins 3

2.1 Acteurs . . . 3
2.2 Besoins fonctionnels . . . 3
2.2.1 Gestion des référentiels . . . 3
2.2.2 Candidatures . . . 3
2.2.3 Notifications . . . 4
2.2.4 Suivi des stages . . . 4
2.3 Besoins non fonctionnels . . . 4
2.4 Cas d'utilisation . . . 4
2.5 Statuts manipulés . . . 5

3 Architecture orientée services 7

3.1 Vue d'ensemble . . . 7
3.2 Rôle de chaque composant . . . 8
3.3 Principes retenus . . . 8
3.4 Interactions entre services . . . 9
3.5 Deux modes d'orchestration . . . 9
3.6 Parcours d'une candidature . . . 9
3.6.1 Trace réelle dans les journaux . . . 10
3.7 Modèle de données . . . 11

4 Documentation des services SOAP 13

4.1 Principes communs . . . 13
4.2 Opérations . . . 13
4.2.1 S1 Gestion des étudiants . . . 14
4.2.2 S2 Gestion des entreprises . . . 14

4.2.3 S3 Gestion des offres de stage . . . 15
4.3 Extrait de WSDL . . . 16
4.4 Exemples d'échanges . . . 17
4.4.1 Vérification d'un étudiant suspendu . . . 17
4.4.2 Faute typée . . . 18
4.5 Implémentation . . . 18

5 Documentation des API REST 20
5.1 Conventions . . . 20
5.2 Points d'accès . . . 21
5.2.1 S4 Gestion des candidatures . . . 21
5.2.2 S5 Notifications . . . 23
5.2.3 S6 Suivi des stages . . . 23
5.3 Exemples d'échanges . . . 24
5.4 Référence OpenAPI et Swagger UI . . . 27
5.5 Implémentation . . . 27

6 Règles de gestion 29
6.1 Vue d'ensemble . . . 29
6.2 Commentaires . . . 30
6.2.1 Pourquoi un contrôle en deux temps pour R5 et R6 . . . 30
6.2.2 R7 : compatibilité et score indicatif . . . 30
6.2.3 R8 et R9 : décision de conception . . . 31
6.2.4 R10 : cohérence entre trois services . . . 31
6.2.5 R11 : boucle de complément . . . 31
6.3 Couverture par les tests . . . 31

7 Processus BPMN d'orchestration 32
7.1 Vue d'ensemble . . . 32
7.2 Déclenchement et données du processus . . . 34
7.3 Étapes du processus . . . 34
7.4 Passerelles de décision . . . 36
7.5 Tâche humaine et boucle de complément . . . 36
7.6 Gestion des erreurs et des indisponibilités . . . 37
7.6.1 Dans le modèle . . . 37
7.6.2 Dans la version déployée . . . 37
7.7 Connecteurs du modèle . . . 38
7.7.1 Connecteurs SOAP . . . 38
7.7.2 Connecteurs REST . . . 38

7.7.3 Mode secours . . . 38
7.8 Exécution et vérification . . . 39

8 Sécurité et gestion des erreurs 40
8.1 Sécurité des échanges SOAP . . . 40
8.2 Sécurité des API REST . . . 40
8.3 Sécurité du portail . . . 41
8.4 Données et configuration . . . 41
8.5 Activer TLS en production . . . 41
8.6 Gestion des erreurs . . . 42
8.6.1 Principes . . . 42
8.6.2 Indisponibilité d'un service . . . 42
8.6.3 Traçabilité . . . 42

9 Scénarios de test et résultats 44
9.1 Stratégie . . . 44
9.2 Scénarios fonctionnels . . . 44
9.3 Tests JUnit . . . 46
9.4 Tests SoapUI . . . 46
9.5 Tests Postman et Newman . . . 47
9.6 Interface dans un navigateur . . . 47
9.7 Reproduire les résultats . . . 48

10 Interface graphique 49
10.1 Principes . . . 49
10.2 Espace étudiant . . . 49
10.3 Espace entreprise . . . 51
10.4 Espace responsable . . . 51
10.5 Vue BPMN en direct . . . 53

11 Déploiement et guide d'exécution 54
11.1 Prérequis . . . 54
11.2 Démarrage . . . 54
11.3 Mode d'orchestration . . . 55
11.4 Configuration . . . 55
11.5 Opérations courantes . . . 56
11.6 Dépannage . . . 56

12 Limites, améliorations et conclusion **57**

12.1 Limites connues 57

12.2 Améliorations possibles 58

12.3 Conclusion 58

Table des figures

2.1 Diagramme des cas d'utilisation 5

3.1 Architecture de la plateforme 7
3.2 Diagramme de séquence : dépôt puis acceptation d'une candidature 10
3.3 Modèle de données, un schéma par service 11

5.1 Swagger UI de S4, servi par le service lui-même 27

7.1 Processus « Candidature » : diagramme BPMN (rendu par bpmn-js depuis `processus-candidature.bpmn`) 33
7.2 Le processus « Candidature » 1.0 déployé et activé dans Bonita (application d'administration) 34
7.3 Dans Bonita : la tâche « Décision de l'entreprise » en attente, avec son minuteur de relance 37
7.4 Dans Bonita : la tâche « Vérification de l'offre » en erreur pendant une panne de S3 38
7.5 Dans Bonita : le cas d'une candidature acceptée, terminé et archivé 39

9.1 SoapUI : suite « S1 Étudiants » exécutée 47
9.2 SoapUI : requête `verifierEtudiant` avec en-tête WS-Security 47
9.3 Postman : exécution de la collection 47
9.4 Rapport HTML de Newman 48

10.1 Connexion, avec les comptes de démonstration 49
10.2 Offres de stage et compatibilité avec le profil 50
10.3 Dossier d'une candidature acceptée : processus BPMN en direct et historique 50
10.4 Candidature reçue : profil masqué et ouverture de l'évaluation 51
10.5 Tableau de bord du responsable des stages 52
10.6 Candidature refusée : branche du refus colorée sur le diagramme 52

Liste des tableaux

1.1 Technologies retenues 2

2.1 Acteurs du système 3
2.2 Besoins non fonctionnels 4
2.3 Cycles de vie des principaux objets 5

3.1 Composants et responsabilités 8
3.2 Matrice des interactions 9

4.1 Fautes SOAP communes aux trois services 13
4.2 Opérations SOAP de S1 Étudiants (port 8081, 7 opérations) 14
4.3 Opérations SOAP de S2 Entreprises (port 8082, 7 opérations) 15
4.4 Opérations SOAP de S3 Offres (port 8083, 8 opérations) 15

5.1 Codes HTTP utilisés 20
5.2 Points d'accès REST de S4 Candidatures (base `/api/v1`, 23 opérations) 21
5.3 Points d'accès REST de S5 Notifications (base `/api/v1`, 4 opérations) 23
5.4 Points d'accès REST de S6 Suivi des stages (base `/api/v1`, 11 opérations) . . . 23

6.1 Règles de gestion R1 à R11 29
6.2 Tests JUnit qui vérifient chaque règle 31

7.1 Contrat d'instanciation du processus 34
7.2 Tâches du processus et services appelés 35
7.3 Conditions des passerelles 36

8.1 Refus d'authentification SOAP 40
8.2 Traitement d'une indisponibilité 42

9.1 Niveaux de test 44
9.2 Scénarios, résultats attendus et obtenus 45
9.3 Résultats JUnit par module (356 tests) 46
9.4 Résultats SoapUI (546 assertions) 46

11.1 Logiciels nécessaires 54
11.2 Adresses utiles 55
11.3 Comptes de démonstration (tous les étudiants et entreprises ont un compte) 55
11.4 Principales variables de configuration 55

11.5 Problèmes fréquents 56

12.1 Pistes d'amélioration 58

CHAPITRE 1

Introduction et contexte

1.1 Contexte

Les établissements d'enseignement supérieur gèrent chaque année des centaines d'offres de stage et des milliers de candidatures. À l'INPT, comme dans beaucoup d'écoles, ces échanges passent encore largement par des courriels et des tableurs : l'entreprise publie une offre, les étudiants envoient leur dossier, le service des stages vérifie à la main l'éligibilité de chacun, relance les entreprises et reporte les décisions. Les informations sont dispersées, les règles appliquées de façon inégale et le suivi d'un dossier repose sur la mémoire des personnes.

Le sujet du projet propose de remplacer ce fonctionnement par une plateforme fondée sur une **architecture orientée services** (SOA). Chaque domaine du métier (étudiants, entreprises, offres, candidatures, notifications, suivi des stages) devient un service autonome, qui expose un contrat et garde ses propres données. Un **processus BPMN** orchestre ces services pour traiter une candidature de bout en bout, du dépôt jusqu'à la décision finale.

1.2 Objectifs

Le cahier des charges fixe les livrables suivants :

- trois services **SOAP** décrits par WSDL : gestion des étudiants (S1), des entreprises (S2) et des offres de stage (S3) ;
- trois services **REST** : gestion des candidatures (S4), notifications (S5) et suivi des stages (S6) ;
- un **processus BPMN** modélisé et exécuté dans Bonita Studio, qui orchestre les appels SOAP et REST, gère les décisions, les erreurs et les notifications ;
- un **rapport technique** et une **vidéo de démonstration** ;
- en bonus, une **interface graphique** pour les étudiants, les entreprises et le responsable des stages.

Nous avons ajouté trois exigences de qualité, qui guident les choix décrits dans la suite : chaque service doit pouvoir être testé seul, chaque règle de gestion doit être vérifiée par un test automatique, et aucune panne d'un service ne doit laisser une candidature dans un état incohérent.

1.3 Choix techniques

Nous avons suivi les technologies recommandées par l'énoncé :

Domaine	Choix
Langage	Java 17, projet Maven multi-module (un module par service, plus un socle commun)
SOAP	JAX-WS (Eclipse Metro 4), WSDL écrits d'abord (*contract first*), clients générés par `wsimport`
REST	JAX-RS (Eclipse Jersey 3.1) sur un serveur Grizzly, contrats OpenAPI 3 écrits à la main
Persistance	MySQL 8.4 dans Docker, un schéma et un compte par service, JDBC avec le pool HikariCP
Orchestration	Bonita Studio Community 2025.2 ; mode de secours « direct » exécuté en Java
Courriels	SMTP vers Mailpit, serveur de courriel de test (aucun envoi réel)
Tests	JUnit 5.11, SoapUI 5.10, Postman et Newman, test de bout en bout en Node.js, Playwright pour l'interface
Interface	HTML, CSS et JavaScript sans framework ; bpmn-js pour l'affichage du processus

Tableau 1.1 – Technologies retenues

Chaque service est un programme indépendant, lancé dans son propre processus Java et à l'écoute sur son propre port. Aucun service ne lit la base d'un autre : toute information extérieure passe par un appel de service. C'est ce qui rend l'architecture réellement orientée services, et non un monolithe découpé en paquets.

1.4 Organisation du rapport

Le chapitre 2 analyse les besoins et les acteurs. Le chapitre 3 décrit l'architecture et les interactions entre services, avec le parcours complet d'une candidature. Les chapitres 4 et 5 documentent les opérations SOAP et les API REST. Le chapitre 6 détaille les règles de gestion R1 à R11, et le chapitre 7 le processus BPMN. Le chapitre 8 traite de la sécurité et de la gestion des erreurs, le chapitre 9 des scénarios et des résultats de test. Les chapitres 10 et 11 présentent l'interface et le déploiement. Le dernier chapitre discute les limites et les améliorations possibles.

> **Données de démonstration**
>
> Toutes les personnes, entreprises, adresses et numéros cités dans ce rapport sont fictifs. Les courriels ne quittent jamais la machine : ils sont reçus par Mailpit.

CHAPITRE 2

Analyse des besoins

2.1 Acteurs

L'analyse du sujet fait apparaître trois acteurs humains et deux acteurs techniques.

Acteur	Rôle
Étudiant	Consulte les offres ouvertes, vérifie sa compatibilité avec chacune, candidate, suit ses candidatures, peut annuler tant que l'évaluation n'a pas commencé, consulte son stage.
Entreprise partenaire	Publie et gère ses offres, reçoit les candidatures, ouvre l'évaluation, accepte, refuse ou demande un complément d'information.
Responsable des stages	Gère les référentiels (étudiants, entreprises), consulte toutes les candidatures, suit les stages (statut, comptes rendus, évaluations), surveille les notifications et l'état des services.
Processus BPMN (Bonita)	Acteur technique : orchestre le traitement d'une candidature en appelant les six services.
Planificateur	Acteur technique : déclenche chaque jour à 8 h les rappels de date limite.

Tableau 2.1 – Acteurs du système

2.2 Besoins fonctionnels

2.2.1 Gestion des référentiels

- Créer, consulter, modifier et supprimer un étudiant (matricule, nom, prénom, filière, niveau, coordonnées, compétences) ; changer son statut (ACTIF, SUSPENDU, EN_STAGE, DIPLOME) selon des transitions autorisées ; vérifier son existence et son éligibilité.
- Créer, consulter, modifier et supprimer une entreprise (raison sociale, ICE, secteur, adresse, responsable de contact) ; consulter ses offres.
- Créer, modifier, clôturer et supprimer une offre (titre, description, compétences requises, filières acceptées, niveau minimum, durée, date limite, nombre de places) ; vérifier sa disponibilité et sa compatibilité avec un étudiant.

2.2.2 Candidatures

- Déposer une candidature avec une lettre de motivation ; obtenir immédiatement le résultat du contrôle (enregistrée, ou rejetée avec la règle et le motif).
- Consulter ses candidatures, leur historique (date, auteur, commentaire de chaque changement) et les étapes du processus.
- Annuler une candidature lorsque les règles le permettent.
- Pour l'entreprise : ouvrir l'évaluation, puis accepter, refuser ou demander un complément.

2.2.3 Notifications

— Confirmer le dépôt à l'étudiant ; informer l'entreprise d'une nouvelle candidature.
— Notifier l'étudiant d'un rejet (avec le motif), d'une acceptation, d'un refus ou d'une demande de complément.
— Envoyer des rappels aux étudiants compatibles qui n'ont pas encore candidaté à une offre dont la date limite approche.

2.2.4 Suivi des stages

— Créer le dossier de stage d'une candidature acceptée ; consulter son statut et ses dates.
— Enregistrer les comptes rendus périodiques et les évaluations (note sur 20 et commentaire).
— Consulter l'historique des stages d'un étudiant.

2.3 Besoins non fonctionnels

Exigence	Réponse apportée
Autonomie des services	Un processus, un port et un schéma de base par service ; aucun accès direct aux données d'un autre service.
Contrats explicites	WSDL pour S1 à S3, OpenAPI pour S4 à S6, versionnés dans le dépôt ; le code est vérifié contre ces contrats par des tests.
Sécurité	WS-Security UsernameToken (SOAP), clé d'API par client (REST), sessions et rôles au portail, mots de passe hachés (BCrypt), masquage des coordonnées.
Robustesse	Délais d'attente sur tous les appels, réponse 503 explicite, sous-processus d'erreur BPMN, alerte au responsable.
Traçabilité	Historique de chaque changement de statut ; identifiant de corrélation propagé dans tous les appels.
Langue	Messages, erreurs, courriels et interface en français.

Tableau 2.2 – Besoins non fonctionnels

2.4 Cas d'utilisation

La figure 2.1 regroupe les principaux cas d'utilisation. Le dépôt d'une candidature *inclut* son traitement par le processus, qui lui-même inclut les vérifications et l'envoi des notifications.

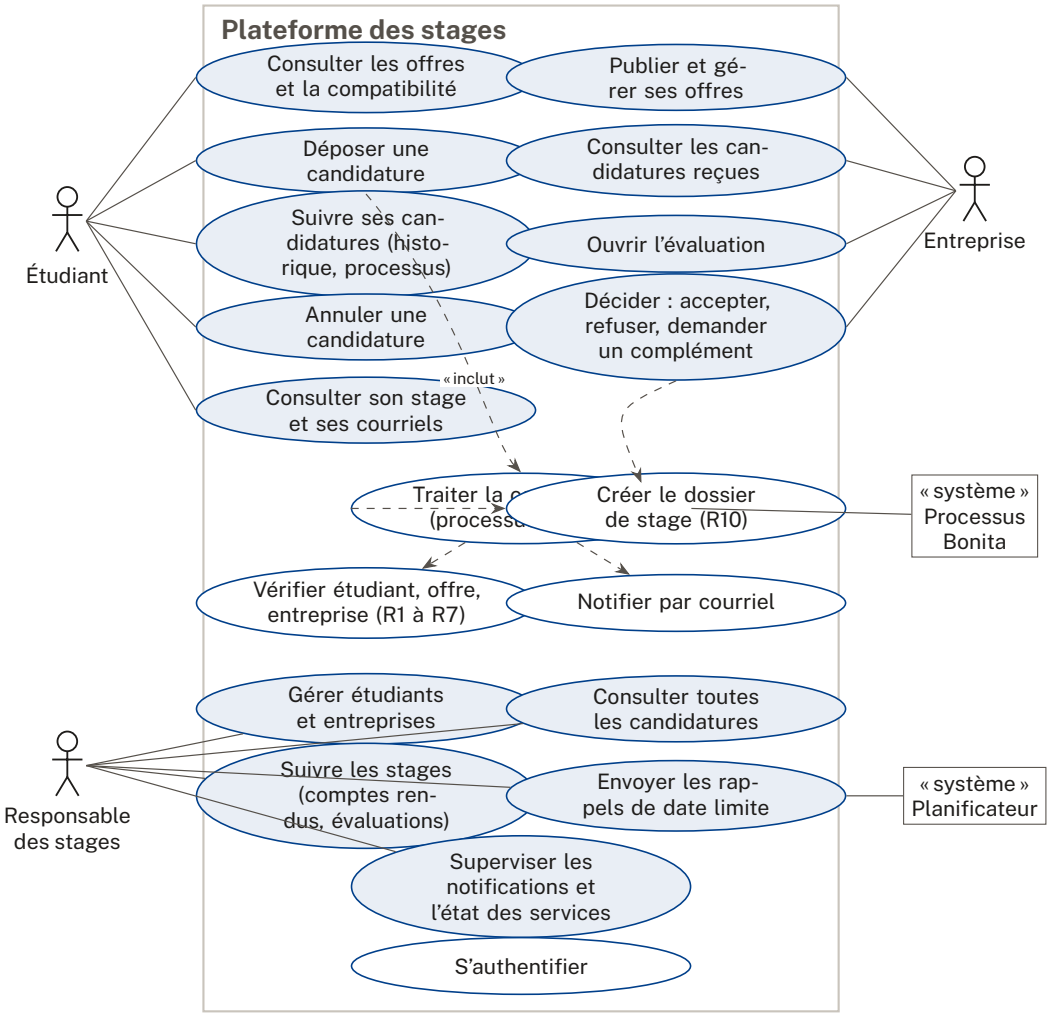


Figure 2.1 – Diagramme des cas d'utilisation

2.5 Statuts manipulés

Quatre cycles de vie structurent le métier. Leurs transitions sont contrôlées par les services eux-mêmes : une transition non prévue est refusée avec une erreur de conflit (faute `ConflitFault` en SOAP, code HTTP 409 en REST).

Objet	Statuts et transitions autorisées
Étudiant (S1)	ACTIF → SUSPENDU, EN_STAGE, DIPLOME ; SUSPENDU → ACTIF ; EN_STAGE → ACTIF, DIPLOME
Offre (S3)	BROUILLON, ACTIVE, POURVUE (plus aucune place), CLOTUREE
Candidature (S4)	DEPOSEE → EN_COURS_EVALUATION → ACCEPTEE ou REFUSEE ; DEPOSEE → ANNULEE (règle R9)
Stage (S6)	PLANIFIE → EN_COURS ou INTERROMPU ; EN_COURS → TERMINE ou INTERROMPU

Tableau 2.3 – Cycles de vie des principaux objets

Une **soumission** (S4) précède la candidature : c'est la demande de l'étudiant telle qu'elle entre dans le processus. Elle devient ENREGISTREE (une candidature est alors créée), REJE-

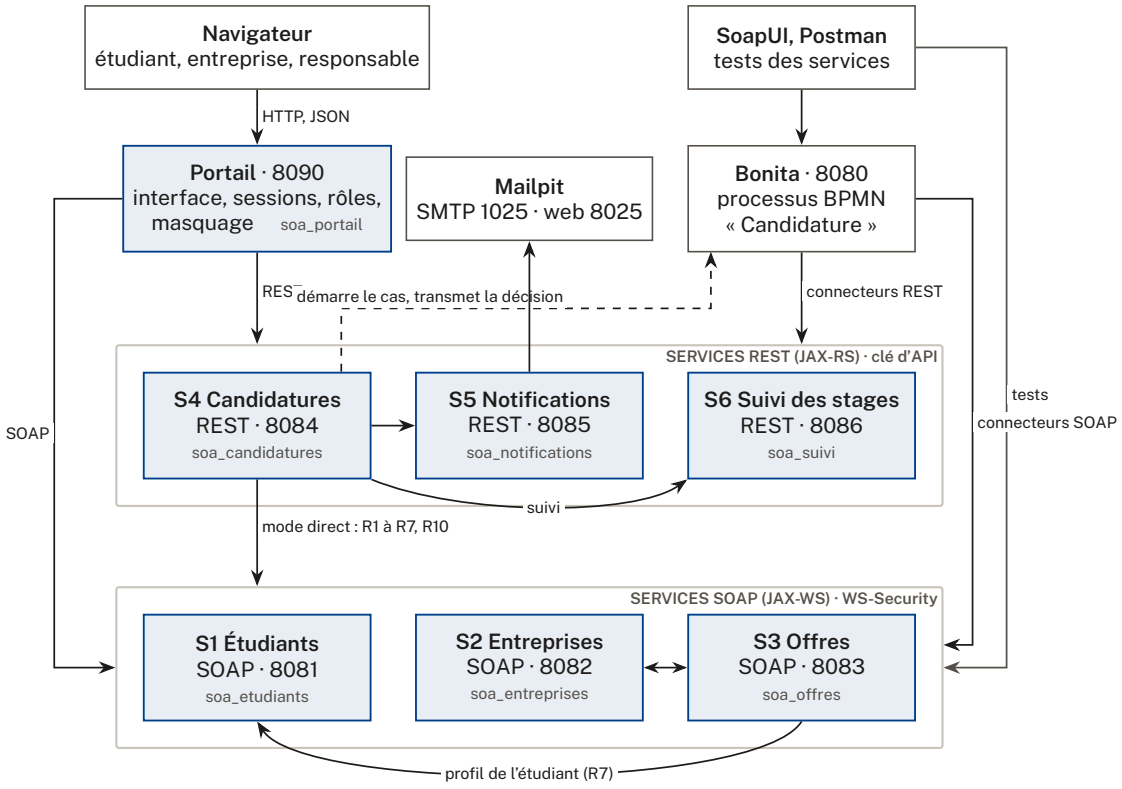
TEE (avec la règle violée et le motif) ou ERREUR (un service n'a pas répondu). Cette distinction garde une trace des dépôts refusés sans les compter comme des candidatures.

CHAPITRE 3

Architecture orientée services

3.1 Vue d'ensemble

La plateforme est composée de six services métier, d'un orchestrateur et d'un portail (figure 3.1). Chaque service est un processus Java distinct, démarré seul, avec son port et son schéma de base de données. Les services de référentiel (S1 à S3) exposent des opérations SOAP ; les services de traitement (S4 à S6) exposent des ressources REST.



Une base MySQL 8.4 (Docker, port 3307) : un schéma et un compte par service, nommés sous chaque service.
Tous les appels portent l'identifiant de corrélation ; en mode bonita, S4 ne vérifie rien lui-même : le processus appelle les services.

Figure 3.1 – Architecture de la plateforme

3.2 Rôle de chaque composant

Composant	Style	Port	Responsabilité
S1 Étudiants	SOAP	8081	Référentiel des étudiants ; vérification d'éligibilité (R1) ; statut EN_STAGE (R10).
S2 Entreprises	SOAP	8082	Référentiel des entreprises et de leur contact ; vérification (R2) ; offres d'une entreprise.
S3 Offres	SOAP	8083	Offres, disponibilité (R2 à R4), compatibilité avec un étudiant (R7), retrait atomique d'une place (R10).
S4 Candidatures	REST	8084	Soumissions, candidatures, historique, journal des étapes ; règles R5, R6, R8, R9 ; déclenche le processus.
S5 Notifications	REST	8085	Courriels HTML (gabarits en français), historique d'envoi, rappels quotidiens de date limite.
S6 Suivi des stages	REST	8086	Dossiers de stage, comptes rendus, évaluations, historique.
Bonita	REST (API Bonita)	8080	Exécute le processus BPMN « Candidature » et appelle les six services par ses connecteurs.
Portail	HTTP, JSON	8090	Interface web, authentification, rôles, masquage, composition des réponses de plusieurs services.

Tableau 3.1 – Composants et responsabilités

3.3 Principes retenus

Contrat d'abord. Les trois WSDL et les trois contrats OpenAPI ont été écrits avant le code. Les classes Java des services SOAP et de leurs clients sont générées à partir des WSDL par `wsimport` : un service ne peut pas s'écarter de son contrat sans que la compilation échoue. Pour REST, un test compare les routes réellement exposées par Jersey avec celles du contrat OpenAPI, dans les deux sens.

Une base par service. MySQL héberge sept schémas (`soa_etudiants`, `soa_entreprises`, `soa_offres`, `soa_candidatures`, `soa_notifications`, `soa_suivi`, `soa_portail`). Chaque service se connecte avec un compte qui n'a de droits que sur son schéma, et seulement en lecture et écriture des données (pas de modification de structure). Une jointure entre deux domaines est donc impossible : le service S4 qui veut le titre d'une offre doit appeler S3.

Couplage faible. Un service connaît les autres par leur adresse, lue dans la configuration, et par leur contrat. Les adresses, délais, comptes et clés se règlent par variables d'environnement, sans recompilation. Tous les appels sortants ont un délai de connexion de 2 s et un délai de lecture de 5 s (SOAP) ou 8 s (REST) : un service lent ne bloque jamais indéfiniment son appelant.

Corrélation. Chaque requête reçoit un identifiant de corrélation (en-tête HTTP `X-Correlation-Id`, en-tête SOAP `CorrelationId`), créé par le premier service s'il est

absent, puis transmis à tous les appels qui en découlent et écrit dans chaque ligne de journal.

3.4 Interactions entre services

Le tableau 3.2 indique, pour chaque appelant, les services sollicités et les opérations utilisées.

Appelant	Appelé	Opérations et motif
S4 (mode direct) ou Bonita	S1	`verifierEtudiant` (R1), `consulterEtudiant`, `changerStatutEtudiant` (R10)
	S2	`verifierEntreprise` (R2), `consulterEntreprise`
	S3	`verifierDisponibilite` (R2 à R4), `consulterOffre`, `verifierCompatibilite` (R7), `decrementerPlaces` (R10)
	S4	contrôles R5 et R6, enregistrement, statut (appels du processus Bonita)
	S5	`POST /notifications` (dépôt, rejet, entreprise, résultat, complément, alerte)
	S6	`POST /stages` (R10)
S2	S3	offres d'une entreprise (`consulterOffresEntreprise`)
S3	S1, S2	profil de l'étudiant pour la compatibilité, entreprise d'une offre
S5 (rappels)	S1, S3, S4	étudiants actifs, offres proches de leur date limite, candidatures déjà déposées
Portail	S1 à S6	toutes les opérations de consultation et de gestion des trois espaces

Tableau 3.2 – Matrice des interactions

3.5 Deux modes d'orchestration

Le processus de référence est le processus BPMN exécuté par Bonita (chapitre 7). S4 le déclenche à chaque dépôt et lui transmet la décision de l'entreprise. Pour le développement et les tests automatiques, qui ne peuvent pas dépendre d'un logiciel de bureau, S4 propose un second mode, `direct`, qui exécute en Java les mêmes étapes, dans le même ordre, avec les mêmes appels. Le mode se choisit au démarrage (`ORCHESTRATION_MODE=bonita` ou `direct`) ; il est `bonita` par défaut.

Les deux modes écrivent le même journal d'étapes (table `etape_processus`), dont les noms sont les identifiants des nœuds du diagramme BPMN. Le portail colore donc le diagramme de la même façon quel que soit le mode, et le test de bout en bout passe à l'identique dans les deux.

3.6 Parcours d'une candidature

La figure 3.2 montre les appels déclenchés par le dépôt d'une candidature puis par son acceptation. L'orchestrateur est S4 en mode direct ou Bonita en mode bonita : les appels aux services sont les mêmes.

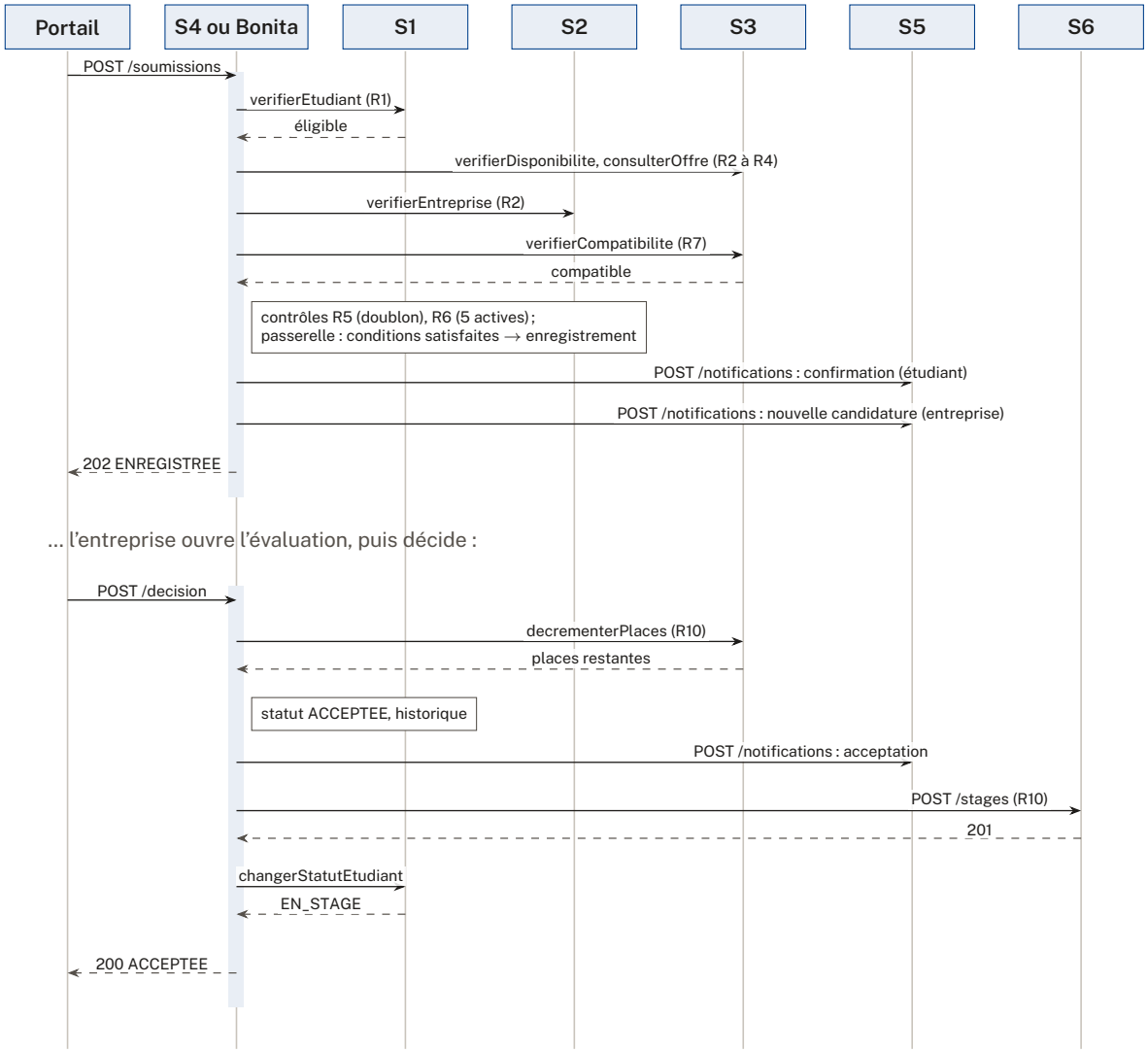


Figure 3.2 – Diagramme de séquence : dépôt puis acceptation d'une candidature

3.6.1 Trace réelle dans les journaux

Le listing 3.1 reproduit les lignes de journal de tous les services qui portent l'identifiant de corrélation d'un même dossier : dépôt, ouverture de l'évaluation et acceptation. Il a été produit par le script `rapport/outils/capturer_exemples.py`, sur la pile en mode direct, sans retouche autre que le regroupement des fichiers par ordre chronologique (colonnes : heure, service, message).

Listing 3.1 – Une candidature traverse les six services (identifiant de corrélation commun)

```
00:59:52.276 soap-etudiants     m.i.s.c.s.GestionnaireSecuriteServeur - Appel SOAP verifierEtudiant par le compte services
00:59:52.381 soap-etudiants     m.i.s.c.s.GestionnaireSecuriteServeur - Appel SOAP consulterEtudiant par le compte services
00:59:52.505 soap-offres        m.i.s.c.s.GestionnaireSecuriteServeur - Appel SOAP verifierDisponibilite par le compte services
00:59:52.615 soap-offres        m.i.s.c.s.GestionnaireSecuriteServeur - Appel SOAP consulterOffre par le compte services
00:59:52.728 soap-entreprises   m.i.s.c.s.GestionnaireSecuriteServeur - Appel SOAP verifierEntreprise par le compte services
00:59:52.830 soap-entreprises   m.i.s.c.s.GestionnaireSecuriteServeur - Appel SOAP consulterEntreprise par le compte services
00:59:52.924 soap-offres        m.i.s.c.s.GestionnaireSecuriteServeur - Appel SOAP verifierCompatibilite par le compte services
00:59:53.038 soap-etudiants     m.i.s.c.s.GestionnaireSecuriteServeur - Appel SOAP consulterEtudiant par le compte services
00:59:53.580 rest-notifications m.i.s.n.ServiceNotifications - Notification CONFIRMATION_DEPOT n° 1 à salma.elidrissi@inpt.ac.ma : ENVOYEE
00:59:53.581 rest-notifications acces - POST /api/v1/notifications -> 201 (460 ms, client services)
00:59:53.630 rest-notifications m.i.s.n.ServiceNotifications - Notification NOUVELLE_CANDIDATURE_ENTREPRISE n° 2 à contact@novapay-maghreb.
      ↪ ma : ENVOYEE
00:59:53.631 rest-notifications acces - POST /api/v1/notifications -> 201 (33 ms, client services)
00:59:53.640 rest-candidatures  acces - POST /api/v1/soumissions -> 202 (1518 ms, client postman)
01:00:04.892 rest-candidatures  acces - POST /api/v1/candidatures/13/evaluation -> 200 (24 ms, client postman)
01:00:09.100 soap-etudiants     m.i.s.c.s.GestionnaireSecuriteServeur - Appel SOAP verifierEtudiant par le compte services
01:00:09.198 soap-etudiants     m.i.s.c.s.GestionnaireSecuriteServeur - Appel SOAP consulterEtudiant par le compte services
```

```
01:00:09.293 soap-offres        m.i.s.c.s.GestionnaireSecuriteServeur - Appel SOAP verifierDisponibilite par le compte services
01:00:09.388 soap-offres        m.i.s.c.s.GestionnaireSecuriteServeur - Appel SOAP consulterOffre par le compte services
01:00:09.485 soap-entreprises   m.i.s.c.s.GestionnaireSecuriteServeur - Appel SOAP verifierEntreprise par le compte services
01:00:09.591 soap-entreprises   m.i.s.c.s.GestionnaireSecuriteServeur - Appel SOAP consulterEntreprise par le compte services
01:00:09.687 soap-offres        m.i.s.c.s.GestionnaireSecuriteServeur - Appel SOAP decrementerPlaces par le compte services
01:00:09.809 rest-notifications acces - POST /api/v1/notifications -> 201 (64 ms, client services)
01:00:09.809 rest-notifications m.i.s.n.ServiceNotifications - Notification ACCEPTATION n° 4 à salma.elidrissi@inpt.ac.ma : ENVOYEE
01:00:09.860 rest-suivi         acces - POST /api/v1/stages -> 201 (39 ms, client services)
01:00:09.960 soap-etudiants     m.i.s.c.s.GestionnaireSecuriteServeur - Appel SOAP changerStatutEtudiant par le compte services
01:00:09.981 rest-candidatures  acces - POST /api/v1/candidatures/13/decision -> 200 (986 ms, client postman)
```

On y lit l'ordre exact des étapes : vérifications SOAP auprès de S1, S3 et S2, enregistrement, deux notifications, réponse 202 au bout de quelques centaines de millisecondes ; puis, lors de la décision, retrait de la place dans S3, courriel d'acceptation, création du stage dans S6 et passage de l'étudiante au statut EN_STAGE dans S1. Chaque appel SOAP est attribué au compte de service qui l'a fait (`services`).

3.7 Modèle de données

Chaque service possède ses tables (figure 3.3). Les identifiants d'un autre domaine (`etudiant_id` dans une candidature, par exemple) sont des références logiques, sans clé étrangère, puisqu'ils désignent des données d'une autre base : leur cohérence est garantie par les vérifications du processus.



Figure 3.3 – Modèle de données, un schéma par service

Trois choix de modélisation portent des règles de gestion directement dans la base, en plus

des contrôles applicatifs :

- **R5** : une colonne calculée `non_annulee` vaut 1 pour une candidature non annulée et `NULL` sinon ; l'index unique (`etudiant_id`, `offre_id`, `non_annulee`) interdit deux candidatures actives pour le même couple, tout en permettant de recandidater après une annulation (MySQL accepte plusieurs `NULL` dans un index unique) ;
- **R6** : l'enregistrement prend un verrou nommé par étudiant (`GET_LOCK`) avant de compter ses candidatures actives, pour que deux dépôts simultanés ne dépassent pas la limite de cinq ;
- **R10** : le retrait d'une place est une seule requête `UPDATE` conditionnelle (places > 0), qui passe l'offre à POURVUE quand le compteur atteint zéro : deux acceptations simultanées ne peuvent pas consommer la même place.

La table `soa_portail.session` ne conserve que l'empreinte SHA-256 du jeton de session, et `compte` le hachage BCrypt du mot de passe : une fuite de la base ne donne accès à aucune session ni à aucun mot de passe.

CHAPITRE 4

Documentation des services SOAP

4.1 Principes communs

Les trois services de référentiel suivent le même modèle.

— **Contrat d'abord.** Chaque WSDL (style *document/literal*, SOAP 1.1) est écrit à la main et documenté opération par opération (`wsdl:documentation`) ; il fait foi. Les classes Java sont générées à la compilation par `wsimport`, et l'implémentation renvoie au fichier par l'attribut `wsdlLocation` de `@WebService` : le WSDL publié par le service est exactement celui du dépôt.
— **Espaces de noms.** `http://stages.inpt.ma/etudiants`, `…/entreprises`, `…/offres` ; l'en-tête de corrélation est dans `http://stages.inpt.ma/commun`.
— **Adresses.** `http://localhost:8081/ws/etudiants`, `:8082/ws/entreprises`, `:8083/ws/offres` ; le WSDL s'obtient en ajoutant `?wsdl` (lecture sans authentification).
— **Validation.** Les données reçues sont validées (Bean Validation : formats du matricule, de l'ICE, du téléphone, longueur des champs, valeurs énumérées) avant tout accès à la base ; chaque champ invalide produit un message en français dans le détail de la faute.
— **Fautes typées.** Toute erreur prévue est une faute déclarée dans le WSDL, avec un détail structuré : un `code` stable, exploitable par un programme, un `message` en français et une liste de `details`. Aucune trace d'exécution Java n'est renvoyée.
— **Sécurité.** Chaque appel doit porter un en-tête WS-Security *UsernameToken* (section 8.1).

Faute	Situation	Exemples de codes
`*IntrouvableFault`	L'identifiant ne correspond à aucun étudiant, entreprise ou offre.	`ETUDIANT_INTROUVABLE`
`DonneesInvalidesFault`	Données absentes ou mal formées.	`DONNEES_INVALIDES`
`ConflitFault`	L'opération contredit l'état ou une règle de gestion.	`TRANSITION_INTERDITE`, `PLUS_DE_PLACE`
`ServiceIndisponibleFault`	La base ou un service dépendant ne répond pas.	`SERVICE_OFFRES_INDISPONIBLE`
`ErreurInterneFault`	Erreur inattendue ; le détail technique reste dans les journaux.	`ERREUR_INTERNE`

Tableau 4.1 – Fautes SOAP communes aux trois services

4.2 Opérations

Les tableaux suivants sont produits automatiquement à partir des WSDL du dépôt (script `rapport/outils/generer_tables.py`) : ils ne peuvent pas diverger du contrat. La dernière

colonne indique les fautes métier attendues pour chaque opération ; toutes peuvent en plus renvoyer `ServiceIndisponibleFault` et `ErreurInterneFault`.

4.2.1 S1 Gestion des étudiants

Le service gère le profil des étudiants et leurs compétences déclarées. `verifierEtudiant` est l'opération appelée par le processus à l'étape 2 : elle ne lève pas de faute pour un étudiant inconnu, mais répond `existe=false`, ce qui permet au processus de rejeter proprement la candidature (règle R1) au lieu de partir en erreur technique.

Tableau 4.2 : Opérations SOAP de S1 Étudiants (port 8081, 7 opérations)

Opération	Entrée	Sortie	Fautes métier
`ajouterEtudiant` Crée un étudiant.	`etudiant` : Etudiant	`etudiantCree` : Etudiant	`DonneesInvalides` `Conflit`
`consulterEtudiant` Renvoie le profil d'un étudiant.	`id` : long	`etudiant` : Etudiant	`EtudiantIntrouvable`
`listerEtudiants` Liste les étudiants selon des critères facultatifs.	`filtre` : FiltreEtudiants, facultatif	`etudiant` : Etudiant[], facultatif	`DonneesInvalides`

`modifierEtudiant`
Remplace le profil. Un changement de statut suit les mêmes règles que changerStatutEtudiant.
	`id` : long
	`etudiant` : Etudiant
		`etudiantModifie` : Etudiant	`EtudiantIntrouvable`
			`DonneesInvalides`
			`Conflit`

`supprimerEtudiant` Supprime un étudiant, sauf pendant un stage.	`id` : long	–	`EtudiantIntrouvable` `Conflit`
`verifierEtudiant` Indique si l'étudiant existe, son statut et s'il peut candidater (R1). Un étudiant inconnu donne existe=false, pas une faute.	`id` : long	`verification` : VerificationEtudiant	`ServiceIndisponible`

`changerStatutEtudiant`
Change le statut selon les transitions autorisées.
	`id` : long
	`statut` : ACTIF | SUSPENDU | EN_STAGE | DIPLOME
		`etudiant` : Etudiant	`EtudiantIntrouvable`
			`DonneesInvalides`
			`Conflit`

4.2.2 S2 Gestion des entreprises

Le service gère les entreprises partenaires et leur responsable de contact. `consulterOffresEntreprise` appelle S3 : S2 ne stocke aucune offre. La suppression d'une entreprise qui a publié des offres est refusée (`ConflitFault`) ; il faut la passer en INACTIVE.

Tableau 4.3 : Opérations SOAP de S2 Entreprises (port 8082, 7 opérations)

Opération	Entrée	Sortie	Fautes métier
`ajouterEntreprise` Crée une entreprise.	`entreprise` : Entreprise	`entrepriseCreee` : Entreprise	`DonneesInvalides` `Conflit`
`consulterEntreprise` Renvoie une entreprise.	`id` : long	`entreprise` : Entreprise	`EntrepriseIntrouvable`
`listerEntreprises` Liste les entreprises selon des critères facultatifs.	`filtre` : FiltreEntreprises, facultatif	`entreprise` : Entreprise[], facultatif	`DonneesInvalides`

`modifierEntreprise`
Remplace les informations, statut compris.
	`id` : long
	`entreprise` : Entreprise
		`entrepriseModifiee` : Entreprise	`EntrepriseIntrouvable`
			`DonneesInvalides`
			`Conflit`

`supprimerEntreprise`
Supprime une entreprise sans offre. Interroge S3.
	`id` : long	–	`EntrepriseIntrouvable`
			`Conflit`
			`ServiceIndisponible`

`verifierEntreprise` Indique si l'entreprise est référencée et active (R2). Une entreprise inconnue donne referencee=false, pas une faute.	`id` : long	`verification` : VerificationEntreprise	`ServiceIndisponible`
`consulterOffresEntreprise` Liste les offres de l'entreprise en appelant S3.	`id` : long	`offre` : OffreResume[], facultatif	`EntrepriseIntrouvable` `ServiceIndisponible`

4.2.3 S3 Gestion des offres de stage

Le service gère les offres et porte trois vérifications du processus : la disponibilité (offre active, date limite, places), la compatibilité avec le profil d'un étudiant (filière acceptée, niveau suffisant, score indicatif de compétences) et le retrait atomique d'une place lors d'une acceptation.

Tableau 4.4 : Opérations SOAP de S3 Offres (port 8083, 8 opérations)

Opération	Entrée	Sortie	Fautes métier
`creerOffre` Crée une offre après avoir vérifié l'entreprise auprès de S2.	`offre` : Offre	`offreCreee` : Offre	`DonneesInvalides` `ServiceIndisponible`

Opération	Entrée	Sortie	Fautes métier

`modifierOffre`
Remplace l'offre. Le statut suit les transitions autorisées ; l'entreprise ne change pas.
	`id` : long
	`offre` : Offre
		`offreModifiee` : Offre	`OffreIntrouvable`
			`DonneesInvalides`
			`Conflit`

`supprimerOffre` Supprime une offre en BROUILLON. Une offre publiée se clôture.	`id` : long	–	`OffreIntrouvable` `Conflit`
`consulterOffre` Renvoie une offre.	`id` : long	`offre` : Offre	`OffreIntrouvable`
`listerOffres` Liste les offres selon des critères facultatifs.	`filtre` : FiltreOffres, facultatif	`offre` : Offre[], facultatif	`DonneesInvalides`
`verifierDisponibilite` Vérifie que l'offre est active (R2), avant sa date limite (R3) et avec une place (R4). Une offre inconnue donne existe=false, pas une faute.	`offreId` : long	`disponibilite` : Disponibilite	`ServiceIndisponible`

`verifierCompatibilite`
Compare l'offre au profil de l'étudiant obtenu auprès de S1 (R7, R3) et calcule le score indicatif.
	`offreId` : long
	`etudiantId` : long
		`compatibilite` : Compatibilite	`OffreIntrouvable`
			`EtudiantIntrouvable`
			`ServiceIndisponible`

`decrementerPlaces` Retire une place de façon atomique. L'offre passe en POURVUE à zéro (R10).	`offreId` : long	`offre` : Offre	`OffreIntrouvable` `Conflit`

4.3 Extrait de WSDL

Le listing 4.1 montre la définition de `verifierEtudiant` : le type de la réponse, puis l'opération dans le `portType` avec ses messages et ses fautes.

Listing 4.1 – Extrait de `etudiants.wsdl` : opération `verifierEtudiant`

```
<!-- Type de la réponse (extrait de wsdl:types) -->
<xs:complexType name="VerificationEtudiant">
  <xs:sequence>
    <xs:element name="existe" type="xs:boolean"/>
    <xs:element name="statut" minOccurs="0">
      <xs:simpleType>
        <xs:restriction base="xs:string">
          <xs:enumeration value="ACTIF"/>
          <xs:enumeration value="SUSPENDU"/>
          <xs:enumeration value="EN_STAGE"/>
```

```
          <xs:enumeration value="DIPLOME"/>
        </xs:restriction>
      </xs:simpleType>
    </xs:element>
    <xs:element name="eligible" type="xs:boolean"/>
    <xs:element name="regle" type="xs:string" minOccurs="0"/>
    <xs:element name="motif" type="xs:string" minOccurs="0"/>
    <xs:element name="nomComplet" type="xs:string" minOccurs="0"/>
    <xs:element name="email" type="xs:string" minOccurs="0"/>
    <xs:element name="filiere" type="xs:string" minOccurs="0"/>
    <xs:element name="niveau" type="xs:string" minOccurs="0"/>
  </xs:sequence>
</xs:complexType>

<!-- Opération dans le portType -->
<wsdl:operation name="verifierEtudiant">
  <wsdl:documentation>Indique si l'étudiant existe, son statut et s'il peut candidater (R1).
    ↪ Un étudiant inconnu donne existe=false, pas une faute. Fautes attendues :
    ↪ ServiceIndisponibleFault.</wsdl:documentation>
  <wsdl:input message="tns:verifierEtudiantRequete"/>
  <wsdl:output message="tns:verifierEtudiantReponse"/>
  <wsdl:fault name="EtudiantIntrouvableFault" message="tns:EtudiantIntrouvableFault"/>
  <wsdl:fault name="DonneesInvalidesFault" message="tns:DonneesInvalidesFault"/>
  <wsdl:fault name="ConflitFault" message="tns:ConflitFault"/>
  <wsdl:fault name="ServiceIndisponibleFault" message="tns:ServiceIndisponibleFault"/>
  <wsdl:fault name="ErreurInterneFault" message="tns:ErreurInterneFault"/>
</wsdl:operation>
```

4.4 Exemples d'échanges

Les échanges suivants ont été enregistrés sur les services en fonctionnement par `rapport/outils/capturer_exemples.py`.

4.4.1 Vérification d'un étudiant suspendu

Listing 4.2 – Requête `verifierEtudiant` avec en-tête WS-Security et corrélation

```
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/" xmlns:etu="http://
    ↪ stages.inpt.ma/etudiants">
  <soapenv:Header>
    <wsse:Security xmlns:wsse="http://docs.oasis-open.org/wss/2004/01/
    ↪ oasis-200401-wss-wssecurity-secext-1.0.xsd">
      <wsse:UsernameToken>
        <wsse:Username>soapui</wsse:Username>
        <wsse:Password>SoapUI#Ws2026</wsse:Password>
      </wsse:UsernameToken>
    </wsse:Security>
    <cor:CorrelationId xmlns:cor="http://stages.inpt.ma/commun">rapport-0001</
    ↪ cor:CorrelationId>
  </soapenv:Header>
  <soapenv:Body>
    <etu:verifierEtudiant>
      <etu:id>17</etu:id>
    </etu:verifierEtudiant>
  </soapenv:Body>
</soapenv:Envelope>
```

Listing 4.3 – Réponse : l'étudiante existe mais n'est pas éligible (règle R1)

```
<!-- HTTP 200 -->
<S:Envelope xmlns:S="http://schemas.xmlsoap.org/soap/envelope/">
  <S:Body>
    <verifierEtudiantResponse xmlns="http://stages.inpt.ma/etudiants">
      <verification>
        <existe>true</existe>
        <statut>SUSPENDU</statut>
        <eligible>false</eligible>
        <regle>R1</regle>
        <motif>Le compte de l'étudiant est suspendu : il ne peut pas candidater.</motif>
        <nomComplet>Sara Bouzidi</nomComplet>
        <email>sara.bouzidi@inpt.ac.ma</email>
        <filiere>ASEDS</filiere>
        <niveau>2A</niveau>
      </verification>
    </verifierEtudiantResponse>
  </S:Body>
</S:Envelope>
```

4.4.2 Faute typée

Pour un identifiant inconnu, `consulterEtudiant` renvoie la faute déclarée dans le WSDL, avec son détail structuré. Conformément à SOAP 1.1, le statut HTTP d'une faute est 500.

Listing 4.4 – Réponse à `consulterEtudiant(9999)` : `EtudiantIntrouvableFault`

```
<!-- HTTP 500 -->
<S:Envelope xmlns:S="http://schemas.xmlsoap.org/soap/envelope/">
  <S:Body>
    <S:Fault xmlns:ns4="http://www.w3.org/2003/05/soap-envelope">
      <faultcode>S:Server</faultcode>
      <faultstring>Aucun étudiant ne correspond à l'identifiant 9999.</faultstring>
      <detail>
        <etudiantIntrouvableFault xmlns="http://stages.inpt.ma/etudiants">
          <code>ETUDIANT_INTROUVABLE</code>
          <message>Aucun étudiant ne correspond à l'identifiant 9999.</message>
        </etudiantIntrouvableFault>
      </detail>
    </S:Fault>
  </S:Body>
</S:Envelope>
```

Sans en-tête WS-Security, la même requête est refusée avant d'atteindre le service, par une faute `wsse:InvalidSecurity` (tableau 8.1).

4.5 Implémentation

Chaque service SOAP comprend quatre couches : l'*endpoint* JAX-WS généré à partir du WSDL, qui ne fait que convertir les types ; un service métier qui applique les règles et les transitions de statut ; un DAO JDBC ; et des clients SOAP vers les autres services quand c'est nécessaire (S2 vers S3, S3 vers S1 et S2). Les gestionnaires JAX-WS (*handlers*) communs, définis dans le module `common`, vérifient le jeton WS-Security, reprennent l'identifiant de corrélation et journalisent chaque appel avec le compte qui l'a fait.

Le serveur utilisé est celui intégré à la JDK (`com.sun.net.httpserver`) piloté par Metro :

aucun serveur d'applications n'est nécessaire, chaque service démarre en une à deux secondes.

CHAPITRE 5

Documentation des API REST

5.1 Conventions

Les trois services REST partagent les conventions suivantes.

— **Ressources et verbes.** Les URI désignent des ressources au pluriel (`/candidatures`, `/stages/{id}/evaluations`), versionnées sous `/api/v1`. `GET` consulte, `POST` crée ou déclenche une action métier, `PUT` remplace, `PATCH` modifie partiellement (le statut), `DELETE` supprime ou annule.
— **Format.** JSON en UTF-8 dans les deux sens ; dates au format ISO 8601.
— **En-têtes.** `X-API-Key` (obligatoire, une clé par client), `X-Correlation-Id` (facultatif, renvoyé dans la réponse), `X-Acteur` (auteur métier d'une action, par exemple `entreprise:1`, inscrit dans les historiques).
— **Contrats.** Chaque service sert son contrat OpenAPI 3 (`/openapi.yaml`) et une interface Swagger UI (`/docs/`) fonctionnant sans Internet. Un test vérifie que chaque route du code figure dans le contrat et réciproquement.

Code	Code d'erreur	Usage
200	–	Consultation, modification, décision réussies.
201	–	Ressource créée (candidature, notification, stage, compte rendu, évaluation).
202	–	Soumission traitée par le processus : enregistrée ou rejetée (le résultat est dans le corps).
204	–	Annulation ou suppression réussie, sans corps.
400	`REQUETE_INVALIDE`	Requête mal formée : JSON illisible, paramètre de type incorrect.
401	`NON_AUTHENTIFIE`	Clé d'API absente ou invalide.
403	`ACCES_INTERDIT`	Clé valide, mais client non autorisé pour cette opération.
404	`*_INTROUVABLE`	Ressource inexistante (`CANDIDATURE_INTROUVABLE`, `STAGE_INTROUVABLE`…).
409	selon la règle	Conflit avec une règle de gestion (R5, R6, R8, R9, R10).
422	`DONNEES_INVALIDES`	Données bien formées mais invalides (champ manquant, note hors de 0 à 20…).
503	`SERVICE_*_INDISPONIBLE`	Un service dépendant (`SERVICE_OFFRES_INDISPONIBLE`…) ou la base (`BASE_INDISPONIBLE`) ne répond pas.

Tableau 5.1 – Codes HTTP utilisés

Toute erreur a le même corps JSON : `code` (stable), `message` (en français), `details` (liste, par exemple un message par champ invalide), `timestamp` et `path`. Le listing 5.3 en donne un exemple réel.

5.2 Points d'accès

Comme pour SOAP, les tableaux sont produits à partir des contrats OpenAPI (script `generer_tables.py`). Un point d'interrogation marque un paramètre facultatif ; la colonne « Corps » nomme le schéma OpenAPI attendu.

5.2.1 S4 Gestion des candidatures

S4 distingue la *soumission* (la demande qui entre dans le processus) de la *candidature* (créée seulement si la passerelle de décision l'accepte). Les points d'accès marqués « processus » sont appelés par Bonita : contrôles R5 et R6, enregistrement, rejet, journal des étapes.

Tableau 5.2 : Points d'accès REST de S4 Candidatures (base `/api/v1`, 23 opérations)

Verbe	URI et rôle	Paramètres	Corps	Codes
post	`/soumissions` Déposer une candidature (déclenche le processus)	–	`NouvelleSoumission`	202, 400, 401, 403, 422, 503
get	`/soumissions/{id}` Consulter une soumission et ses étapes	`id`	–	200, 401, 404
get	`/soumissions/{id}/etapes` Étapes du processus déjà exécutées (vue BPMN en direct)	`id`	–	200, 401, 404
post	`/soumissions/{id}/etapes` Signaler une étape exécutée par le processus Bonita	`id`	`NouvelleEtape`	201, 400, 401, 403, 404, 422
post	`/soumissions/{id}/rejet` Enregistrer le rejet décidé par le processus Bonita (R1 à R7)	`id`	`Rejet`	200, 400, 401, 403, 404, 409, 422
post	`/soumissions/{id}/erreur` Signaler l'interruption du processus Bonita (service indisponible)	`id`	`JSON`	200, 400, 401, 403, 404, 422
get	`/processus/mode` Mode d'orchestration en service (bonita ou direct)	–	–	200, 401
get	`/etudiants/{id}/soumissions` Soumissions d'un étudiant, y compris rejetées	`id`	–	200, 401, 422
post	`/candidatures` Enregistrer une candidature (appelé par le processus après la passerelle de décision)	–	`NouvelleCandidature`	201, 400, 401, 403, 404, 409, 422, 503

Verbe	URI et rôle	Paramètres	Corps	Codes
get	`/candidatures` Lister les candidatures (filtres combinables)	`statut?`, `etudiantId?`, `offreId?`, `entrepriseId?`	–	200, 400, 401
get	`/candidatures/verification-doublon` Règle R5 - candidature non annulée existante pour ce couple étudiant et offre	`etudiantId`, `offreId`	–	200, 400, 401
get	`/candidatures/{id}` Consulter une candidature	`id`	–	200, 401, 404
delete	`/candidatures/{id}` Annuler une candidature (règle R8)	`id`	–	204, 401, 404, 409
get	`/candidatures/{id}/historique` Historique des changements de statut (date, acteur, commentaire)	`id`	–	200, 401, 404
get	`/candidatures/{id}/etapes` Étapes du processus de la candidature (vue BPMN en direct)	`id`	–	200, 401, 404
patch	`/candidatures/{id}/statut` Changer le statut (règle R9)	`id`	`ChangementStatut`	200, 400, 401, 403, 404, 409, 422
post	`/candidatures/{id}/evaluation` Démarrer l'évaluation (l'entreprise ouvre le dossier)	`id`	–	200, 401, 403, 404, 409
post	`/candidatures/{id}/decision` Décision de l'entreprise (étapes 7 à 9)	`id`	`Decision`	200, 400, 401, 403, 404, 409, 422, 503
post	`/candidatures/{id}/suivi/reprendre` Rejouer les étapes de suivi d'une candidature acceptée (stage, statut de l'étudiant)	`id`	–	200, 401, 403, 404, 409, 503
get	`/etudiants/{id}/candidatures` Candidatures d'un étudiant	`id`	–	200, 401, 422
get	`/etudiants/{id}/candidatures/actives/compte` Règle R6 - nombre de candidatures actives de l'étudiant	`id`	–	200, 401, 422
get	`/offres/{id}/candidatures` Candidatures reçues pour une offre	`id`	–	200, 401, 422
get	`/entreprises/{id}/candidatures` Candidatures reçues par une entreprise	`id`, `statut?`	–	200, 400, 401, 422

5.2.2 S5 Notifications

Chaque notification est enregistrée puis réellement envoyée par SMTP (Mailpit). Le statut ENVOYEE ou ECHEC est conservé avec l'erreur éventuelle ; la réponse est 201 dans les deux cas, car la notification a bien été enregistrée. Huit types sont gérés : confirmation de dépôt, candidature rejetée, nouvelle candidature (pour l'entreprise), acceptation, refus, demande de complément, rappel de date limite, alerte au responsable.

Tableau 5.3 : Points d'accès REST de S5 Notifications (base `/api/v1`, 4 opérations)

Verbe	URI et rôle	Paramètres	Corps	Codes
post	`/notifications` Envoyer une notification	–	`NouvelleNotification`	201, 400, 401, 403, 422, 503
get	`/notifications` Lister les notifications	`destinataire?`, `candidatureId?`, `type?`, `limite?`	–	200, 400, 401, 503
get	`/notifications/{id}` Consulter une notification, avec le corps HTML envoyé	`id`	–	200, 401, 404, 422
post	`/rappels/executer` Exécuter maintenant les rappels de date limite	`jours?`	–	200, 400, 401, 403, 503

5.2.3 S6 Suivi des stages

La création d'un stage est idempotente : un second appel pour la même candidature renvoie le stage existant au lieu d'en créer un doublon, ce qui permet de rejouer sans risque une étape de suivi interrompue.

Tableau 5.4 : Points d'accès REST de S6 Suivi des stages (base `/api/v1`, 11 opérations)

Verbe	URI et rôle	Paramètres	Corps	Codes
post	`/stages` Créer le dossier de stage d'une candidature acceptée (règle R10)	–	`NouveauStage`	200, 201, 400, 401, 409, 422, 503
get	`/stages` Lister les stages	`statut?`, `etudiantId?`, `entrepriseId?`	–	200, 400, 401, 503
get	`/stages/{id}` Consulter un stage (statut et dates)	`id`	–	200, 401, 404, 422

Verbe	URI et rôle	Paramètres	Corps	Codes
patch	`/stages/{id}/statut` Changer le statut d'un stage	`id`	`ChangementStatut`	200, 400, 401, 404, 409, 422
post	`/stages/{id}/comptes-rendus` Enregistrer un compte rendu périodique	`id`	`NouveauCompteRendu`	201, 400, 401, 404, 409, 422
get	`/stages/{id}/comptes-rendus` Lister les comptes rendus d'un stage	`id`	–	200, 401, 404
post	`/stages/{id}/evaluations` Enregistrer une évaluation (note sur 20 et commentaires)	`id`	`NouvelleEvaluation`	201, 400, 401, 404, 409, 422
get	`/stages/{id}/evaluations` Lister les évaluations d'un stage	`id`	–	200, 401, 404
get	`/stages/{id}/historique` Historique des opérations d'un stage	`id`	–	200, 401, 404
get	`/etudiants/{id}/stages` Historique des stages d'un étudiant	`id`	–	200, 401, 422
get	`/candidatures/{id}/stage` Stage créé pour une candidature	`id`	–	200, 401, 404

5.3 Exemples d'échanges

Les échanges suivants ont été enregistrés sur la pile en fonctionnement (mode direct) ; seule l'indentation du JSON a été ajoutée.

Listing 5.1 – Dépôt d'une candidature : soumission enregistrée, avec les étapes du processus

```
POST /api/v1/soumissions
X-Correlation-Id: dossier-20261110-005941
X-API-Key: cle-postman-2026-Rb3WsE
X-Acteur: etudiant:1

{
  "etudiantId": 1,
  "offreId": 1,
  "lettreMotivation": "Étudiante en AMOA, je souhaite contribuer au recueil des besoins de
    ↪ votre application de paiement mobile."
}

--> HTTP 202
{
  "id": 13,
  "etudiantId": 1,
  "offreId": 1,
  "lettreMotivation": "Étudiante en AMOA, je souhaite contribuer au recueil des besoins de
    ↪ votre application de paiement mobile.",
  "statut": "ENREGISTREE",
  "candidatureId": 13,
```

```
  "creeLe": "2026-11-10T00:59:52",
  "etapes": [
    {"etape": "RECEPTION", "resultat": "OK", "detail": "Candidature reçue", "date": "
    ↪ 2026-11-10T00:59:52.167", "acteur": "etudiant:1"},
    {"etape": "VERIFICATION_ETUDIANT", "resultat": "OK", "detail": "Étudiant actif (R1)", "
    ↪ date": "2026-11-10T00:59:52.389", "acteur": "processus"},
    {"etape": "VERIFICATION_OFFRE", "resultat": "OK", "detail": "Offre active, entreprise
    ↪ active, profil compatible (R2, R7)", "date": "2026-11-10T00:59:53.053", "acteur": "
    ↪ processus"},
    {"etape": "CONTROLE_CANDIDATURE", "resultat": "OK", "detail": "Date limite, places,
    ↪ doublon et limite de 5 respectés (R3 à R6)", "date": "2026-11-10T00:59:53.075", "acteur
    ↪ ": "processus"},
    {"etape": "PASSERELLE_DECISION", "resultat": "OK", "detail": "Conditions satisfaites", "
    ↪ date": "2026-11-10T00:59:53.079", "acteur": "processus"},
    {"etape": "ENREGISTREMENT", "resultat": "OK", "detail": "Candidature 13 enregistrée", "
    ↪ date": "2026-11-10T00:59:53.111", "acteur": "processus"},
    {"etape": "NOTIFICATION_DEPOT", "resultat": "OK", "detail": "Courriel CONFIRMATION_DEPOT
    ↪ envoyé à salma.elidrissi@inpt.ac.ma", "date": "2026-11-10T00:59:53.589", "acteur": "
    ↪ processus"},
    {"etape": "TRANSMISSION_ENTREPRISE", "resultat": "OK", "detail": "Courriel
    ↪ NOUVELLE_CANDIDATURE_ENTREPRISE envoyé à contact@novapay-maghreb.ma", "date": "
    ↪ 2026-11-10T00:59:53.635", "acteur": "processus"}
  ]
}
```

Listing 5.2 – Dépôt refusé par la règle R3 (date limite dépassée)

```
POST /api/v1/soumissions
X-Correlation-Id: rapport-0001
X-API-Key: cle-postman-2026-Rb3WsE
X-Acteur: etudiant:20

{
  "etudiantId": 20,
  "offreId": 13,
  "lettreMotivation": "Je souhaite rejoindre votre équipe décisionnelle pour ce stage de
    ↪ première année."
}

--> HTTP 202
{
  "id": 14,
  "etudiantId": 20,
  "offreId": 13,
  "lettreMotivation": "Je souhaite rejoindre votre équipe décisionnelle pour ce stage de
    ↪ première année.",
  "statut": "REJETEE",
  "regleRejetee": "R3",
  "motifRejet": "La date limite de candidature est dépassée (2026-11-05).",
  "creeLe": "2026-11-10T00:59:55",
  "etapes": [
    {"etape": "RECEPTION", "resultat": "OK", "detail": "Candidature reçue", "date": "
    ↪ 2026-11-10T00:59:55.711", "acteur": "etudiant:20"},
    {"etape": "VERIFICATION_ETUDIANT", "resultat": "OK", "detail": "Étudiant actif (R1)", "
    ↪ date": "2026-11-10T00:59:55.943", "acteur": "processus"},
    {"etape": "VERIFICATION_OFFRE", "resultat": "OK", "detail": "Offre active, entreprise
    ↪ active, profil compatible (R2, R7)", "date": "2026-11-10T00:59:56.541", "acteur": "
    ↪ processus"},
    {"etape": "CONTROLE_CANDIDATURE", "resultat": "REJET", "detail": "R3 : La date limite de
```

```
    ↪ candidature est dépassée (2026-11-05).", "date": "2026-11-10T00:59:56.557", "acteur": "
    ↪ processus"},
    {"etape": "PASSERELLE_DECISION", "resultat": "REJET", "detail": "R3 : La date limite de
    ↪ candidature est dépassée (2026-11-05).", "date": "2026-11-10T00:59:56.57", "acteur": "
    ↪ processus"},
    {"etape": "NOTIFICATION_REJET", "resultat": "OK", "detail": "Courriel CANDIDATURE_REJETEE
    ↪ envoyé à aicha.tahiri@inpt.ac.ma", "date": "2026-11-10T00:59:56.621", "acteur": "
    ↪ processus"}
  ]
}
```

Listing 5.3 – Annulation refusée pendant l'évaluation (règle R8)

```
DELETE /api/v1/candidatures/13
X-Correlation-Id: rapport-0001
X-API-Key: cle-postman-2026-Rb3WsE
X-Acteur: etudiant:1

--> HTTP 409
{
  "code": "ANNULATION_IMPOSSIBLE",
  "message": "Règle R8 : une candidature ne peut être annulée que tant qu'elle est déposée (
    ↪ statut actuel : EN_COURS_EVALUATION).",
  "details": [],
  "timestamp": "2026-11-10T01:00:06Z",
  "path": "/api/v1/candidatures/13"
}
```

Listing 5.4 – Décision de l'entreprise : acceptation

```
POST /api/v1/candidatures/13/decision
X-Correlation-Id: dossier-20261110-005941
X-API-Key: cle-postman-2026-Rb3WsE
X-Acteur: entreprise:1

{
  "decision": "ACCEPTER",
  "commentaire": "Profil en adéquation avec la mission."
}

--> HTTP 200
{
  "candidature": {"id": 13, "soumissionId": 13, "etudiantId": 1, "offreId": 1, "entrepriseId":
    ↪  1, "lettreMotivation": "Étudiante en AMOA, je souhaite contribuer au recueil des
    ↪ besoins de votre application de paiement mobile.", "scoreCompetences": 100, "statut": "
    ↪ ACCEPTEE", "creeLe": "2026-11-10T00:59:53", "modifieLe": "2026-11-10T01:00:09"},
  "etapes": [
    {"etape": "DECISION_ENTREPRISE", "resultat": "OK", "detail": "Acceptée : Profil en
    ↪ adéquation avec la mission.", "date": "2026-11-10T01:00:09.7257395", "acteur": "
    ↪ entreprise:1"},
    {"etape": "MISE_A_JOUR_PLACES", "resultat": "OK", "detail": "Place retirée de l'offre (R10
    ↪ )", "date": "2026-11-10T01:00:09.7301431", "acteur": "processus"},
    {"etape": "NOTIFICATION_RESULTAT", "resultat": "OK", "detail": "Courriel ACCEPTATION
    ↪ envoyé à salma.elidrissi@inpt.ac.ma", "date": "2026-11-10T01:00:09.8171537", "acteur":
    ↪ "processus"},
    {"etape": "CREATION_STAGE", "resultat": "OK", "detail": "Dossier de stage créé du
    ↪ 2026-12-10 au 2027-03-04 (R10)", "date": "2026-11-10T01:00:09.8686908", "acteur": "
    ↪ processus"},
    {"etape": "MISE_A_JOUR_ETUDIANT", "resultat": "OK", "detail": "Étudiant passé EN_STAGE (
```

```
      ↪ R10)", "date": "2026-11-10T01:00:09.9796312", "acteur": "processus"}
  ]
}
```

Les autres échanges enregistrés (consultation de la candidature, stage créé dans S6, notification enregistrée dans S5, erreurs 404 et 401) sont dans `rapport/generes/exemples/`.

5.4 Référence OpenAPI et Swagger UI

Les contrats complets, avec les schémas de chaque corps et des exemples pour chaque opération, sont dans le dépôt :

- `rest-candidatures/src/main/resources/openapi.yaml` ;
- `rest-notifications/src/main/resources/openapi.yaml` ;
- `rest-suivi/src/main/resources/openapi.yaml`.

Une fois les services démarrés, chacun est consultable et testable dans Swagger UI (figure 5.1) : `http://localhost:8084/docs/`, `http://localhost:8085/docs/`, `http://localhost:8086/docs/`. Le bouton « Authorize » permet de saisir la clé d'API.

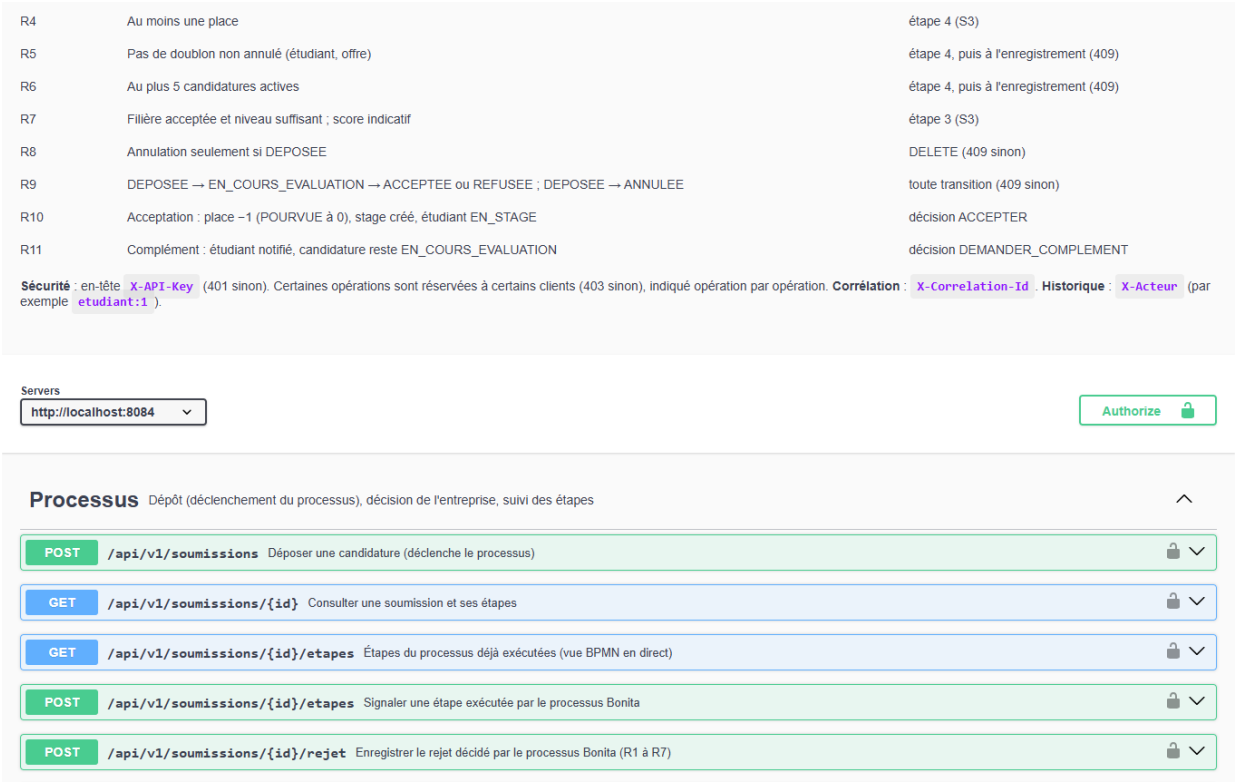


Figure 5.1 – Swagger UI de S4, servi par le service lui-même

5.5 Implémentation

Chaque service REST est une application Jersey sur un serveur Grizzly. Les ressources JAX-RS ne contiennent que la traduction HTTP ; les règles sont dans une classe de service, testée sans serveur. Des filtres communs (module `common`) contrôlent la clé d'API et les clients autorisés, propagent l'identifiant de corrélation et journalisent chaque accès avec sa durée ;

un unique *ExceptionMapper* produit le corps d'erreur uniforme. Les appels sortants passent par un client HTTP commun, avec délais d'attente et traduction des erreurs du service appelé : un 404 de S3 devient une erreur métier claire dans S4, et une panne devient un 503.

CHAPITRE 6

Règles de gestion

Onze règles encadrent le traitement d'une candidature. Chacune porte un identifiant, repris partout : dans le code, les messages d'erreur, le journal des étapes, les courriels de rejet, les tests et l'interface. Un étudiant dont la candidature est refusée voit ainsi la règle exacte et un motif en français, par exemple « R3 : La date limite de candidature est dépassée (2026-11-04) ».

6.1 Vue d'ensemble

Tableau 6.1 : Règles de gestion R1 à R11

Règle	Énoncé	Vérifiée par	En cas de violation
R1	L'étudiant existe et a le statut ACTIF ; un étudiant SUSPENDU, EN_STAGE ou DIPLOME ne peut pas candidater.	Étape 2 : S1 `verifierEtudiant`.	Soumission rejetée, courriel de rejet avec le motif.
R2	L'offre est ACTIVE ; son entreprise est référencée et ACTIVE.	Étape 3 : S3 `verifierDisponibilite`, S2 `verifierEntreprise`.	Soumission rejetée.
R3	La date limite de candidature n'est pas dépassée.	Étape 4 : d'après la disponibilité renvoyée par S3.	Soumission rejetée.
R4	L'offre a au moins une place disponible.	Étape 4 : d'après S3.	Soumission rejetée.
R5	Pas de doublon : une seule candidature non annulée par couple (étudiant, offre).	Étape 4 : S4 `verification-doublon` ; puis index unique en base à l'enregistrement.	Rejet au contrôle ; 409 si deux dépôts simultanés passent le contrôle.
R6	Au plus cinq candidatures actives (DEPOSEE ou EN_COURS_EVALUATION) par étudiant.	Étape 4 : S4 compte des actives ; puis verrou par étudiant à l'enregistrement.	Rejet au contrôle ; 409 à l'enregistrement.

Règle	Énoncé	Vérifiée par	En cas de violation
R7	Compatibilité : la filière de l'étudiant est acceptée par l'offre et son niveau est au moins le niveau minimum. Le score de compétences est seulement indicatif.	Étape 3 : S3 `verifierCompatibilite`.	Soumission rejetée.
R8	Annulation possible seulement tant que la candidature est DEPOSEE.	S4 `DELETE /candidatures/{id}`.	409 `ANNULATION_IMPOSSIBLE`.
R9	Transitions autorisées : DEPOSEE → EN_COURS_EVALUATION → ACCEPTEE ou REFUSEE ; DEPOSEE → ANNULEE.	S4, à chaque changement de statut.	409 `TRANSITION_INTERDITE`.
R10	À l'acceptation : une place de moins ; l'offre devient POURVUE à zéro ; dossier de stage créé dans S6 ; étudiant passé EN_STAGE dans S1.	Étape 9 : S3 `decrementerPlaces`, S6 `POST /stages`, S1 `changerStatutEtudiant`.	409 si plus aucune place ; reprise possible des étapes de suivi après une panne.
R11	Demande de complément : l'étudiant est notifié, la candidature reste EN_COURS_EVALUATION et la décision revient à l'entreprise.	Passerelle de décision et boucle du processus.	– (cas nominal de la boucle).

6.2 Commentaires

6.2.1 Pourquoi un contrôle en deux temps pour R5 et R6

Le contrôle de l'étape 4 suffit pour un dépôt isolé et permet un rejet propre, avec courriel. Mais deux dépôts du même étudiant envoyés au même instant passeraient tous deux le contrôle avant que l'un soit enregistré. L'enregistrement revérifie donc les deux règles là où elles ne peuvent pas être contournées : l'index unique pour R5 et un verrou nommé par étudiant pour R6 (voir le modèle de données, figure 3.3). Le second dépôt reçoit alors une erreur de conflit au lieu de créer une sixième candidature active. Un test d'intégration lance des enregistrements concurrents sur la vraie base pour le vérifier.

6.2.2 R7 : compatibilité et score indicatif

L'offre déclare ses filières acceptées et un niveau minimum (1A, 2A ou 3A). S3 compare ces critères au profil renvoyé par S1. Le score de compétences, pourcentage des compétences

demandées que l'étudiant déclare, est calculé et affiché à l'entreprise comme à l'étudiant, mais il ne bloque jamais une candidature : un étudiant peut combler un écart pendant son stage, et la décision appartient à l'entreprise.

6.2.3 R8 et R9 : décision de conception

Le passage à EN_COURS_EVALUATION est déclenché par l'entreprise elle-même (« Démarrer l'évaluation ») : à partir de ce moment, l'étudiant ne peut plus retirer sa candidature. Ce choix rend la règle R8 explicite pour les deux parties : l'interface de l'étudiant indique pourquoi le bouton d'annulation a disparu.

6.2.4 R10 : cohérence entre trois services

L'acceptation modifie trois services. L'ordre retenu protège la ressource rare : la place est réservée en premier (S3, opération atomique) ; si elle n'est plus disponible, la décision est refusée (409) et rien n'est modifié. Viennent ensuite le statut ACCEPTEE et la notification, puis le dossier de stage (S6) et le statut de l'étudiant (S1). Si l'un de ces deux derniers appels échoue, l'étape est marquée en échec dans le journal, le responsable la voit dans son espace et peut la relancer (« Relancer le suivi ») ; la création du stage étant idempotente, la relance ne crée jamais de doublon.

6.2.5 R11 : boucle de complément

La demande de complément n'est pas une décision finale. Le processus notifie l'étudiant avec le commentaire de l'entreprise, ajoute une ligne à l'historique (statut inchangé) et recrée la tâche de décision. L'entreprise peut demander plusieurs compléments, puis accepter ou refuser.

6.3 Couverture par les tests

Chaque règle est vérifiée à plusieurs niveaux : par des tests unitaires du service qui la porte, par des tests d'intégration sur MySQL pour celles qui reposent sur la base, par la collection Postman et par le test de bout en bout (chapitre 9).

Règle	Classes de test JUnit
R1	`ServiceEtudiantsTest`, `EtudiantsSoapTest`, `OrchestrateurDirectTest`, `RessourceCandidaturesTest`, `CandidatureDaoJdbcIT`
R2	`ServiceEntreprisesTest`, `ServiceOffresTest`, `OrchestrateurDirectTest`
R3, R4	`ServiceOffresTest`, `ServiceCandidaturesTest`, `OrchestrateurDirectTest`
R5, R6	`ServiceCandidaturesTest`, `OrchestrateurDirectTest`, `CandidatureDaoJdbcIT` (enregistrements concurrents)
R7	`ServiceOffresTest`, `OrchestrateurDirectTest`
R8, R9	`ServiceCandidaturesTest`, `RessourceCandidaturesTest`, `OrchestrateurBonitaTest`, `PortailTest`
R10	`ServiceOffresTest`, `OffreDaoJdbcIT`, `ServiceSuiviTest`, `OrchestrateurDirectTest`
R11	`OrchestrateurDirectTest` ; Postman et bout en bout

Tableau 6.2 – Tests JUnit qui vérifient chaque règle

CHAPITRE 7

Processus BPMN d'orchestration

7.1 Vue d'ensemble

Le processus « Candidature » (version 1.0) est modélisé en BPMN 2.0 et exécuté par Bonita Community 2025.2. Il est déclenché à chaque dépôt et couvre les neuf étapes de l'énoncé, de la réception de la candidature à la mise à jour du suivi des stages.

Le diagramme comprend un bassin, trois couloirs (Étudiant, Système, Entreprise), dix-huit tâches de service, une tâche humaine, quatre passerelles exclusives, un minuteur de relance, quatre événements de fin et un sous-processus événementiel de gestion des erreurs. Son fichier BPMN (`bpmn/processus-candidature.bpmn`) est le même que celui qu'affiche le portail : les identifiants des nœuds sont les noms d'étapes que S4 enregistre dans son journal, ce qui permet de colorer le diagramme au fur et à mesure du traitement (chapitre 10).

Le processus existe sous deux formes, qui partagent les mêmes tâches, passerelles et noms :

- **le modèle** (figure 7.1) : fichier BPMN à importer dans Bonita Studio, avec des connecteurs SOAP et REST et le sous-processus d'erreur ; sa construction est décrite pas à pas dans `docs/bonita/GUIDE_BONITA.md` ;
- **la version déployée** : construite par programme avec l'API du moteur Bonita (`docs/bonita/deploiement`), empaquetée en archive `bpmn/Candidature--1.0.bar` et déployée par l'API REST de Bonita (`scripts\deployer-bonita.ps1`). Elle utilise le *mode secours* du guide : chaque tâche de service est une opération Groovy qui appelle elle-même les services, sans connecteur. Les connecteurs SOAP et REST ne sont en effet pas fournis avec Bonita Community 2025.2 ; ils s'ajoutent depuis la Marketplace.

C'est la version déployée qui a été vérifiée de bout en bout face au vrai Bonita (section 7.8).

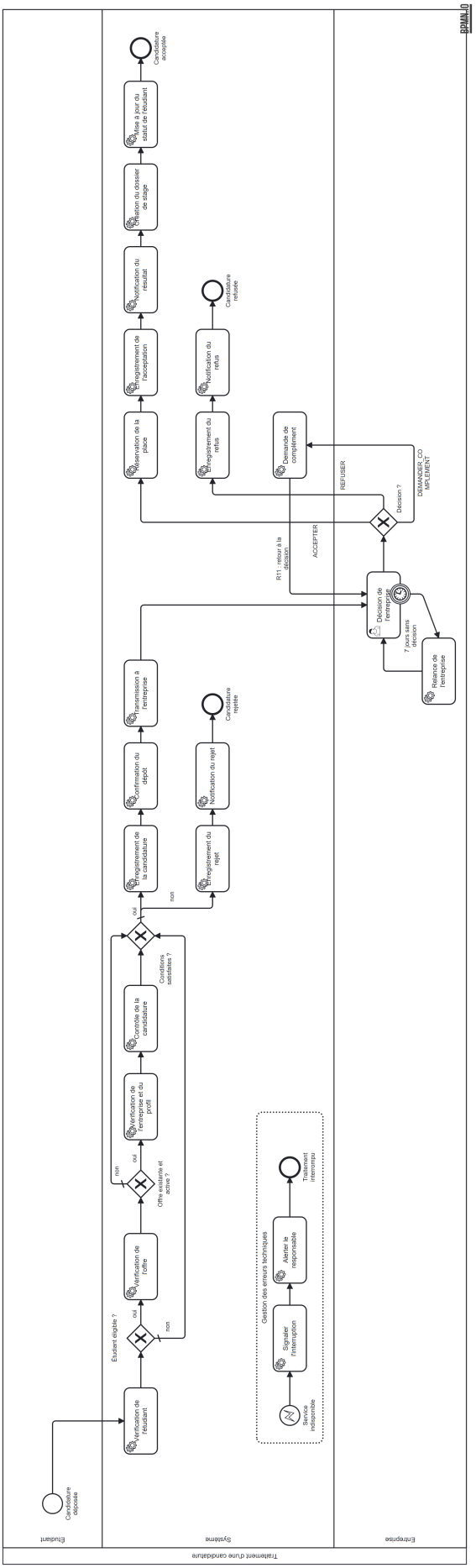


Figure 7.1 – Processus « Candidature » : diagramme BPMN (rendu par bpmn-js depuis `processus-candidature.bpmn`)

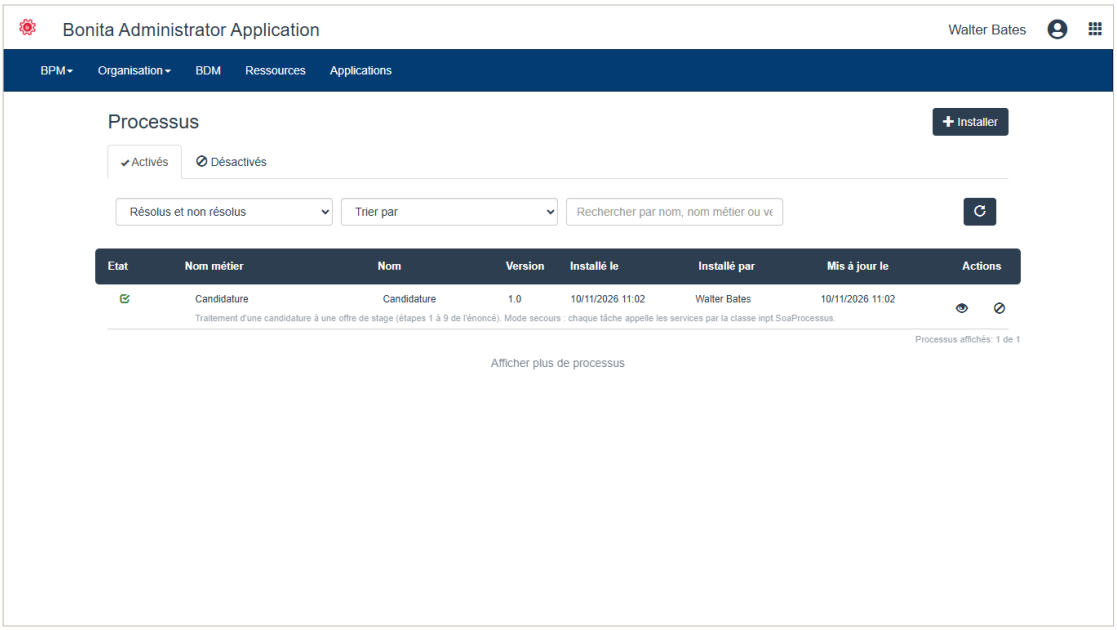


Figure 7.2 – Le processus « Candidature » 1.0 déployé et activé dans Bonita (application d'administration)

7.2 Déclenchement et données du processus

S4 démarre un cas par l'API REST de Bonita : connexion (`/loginservice`, jeton `X-Bonita-API-Token`), recherche du processus par nom et version, puis instanciation avec le contrat suivant.

Entrée du contrat	Type	Rôle
`soumissionId`	LONG	Soumission créée par S4 ; permet au processus d'écrire dans le bon dossier (journal, rejet, enregistrement).
`etudiantId`	LONG	Étudiant qui candidate.
`offreId`	LONG	Offre visée.
`lettreMotivation`	TEXT	Lettre transmise à l'entreprise.

Tableau 7.1 – Contrat d'instanciation du processus

Le processus n'a qu'une variable, `dossier` (une table associative), initialisée à partir du contrat et complétée par chaque tâche (résultat des vérifications, règle violée, identifiant de la candidature, décision…). Avec une seule variable, chaque tâche n'a qu'une opération, et la logique d'appel et de lecture des réponses est regroupée dans une classe Groovy, `inpt.SoaProcessus`, livrée dans l'archive du processus et testée hors de Bonita.

7.3 Étapes du processus

Tableau 7.2 : Tâches du processus et services appelés

Étape	Élément BPMN	Appels	Règles
1	Candidature déposée (début)	déclenché par S4 à la réception	–
2	Vérification de l'étudiant	SOAP S1 `verifierEtudiant`	R1
3	Vérification de l'offre	SOAP S3 `verifierDisponibilite`	R2 (offre)
3	Vérification de l'entreprise et du profil	SOAP S3 `consulterOffre`, S2 `verifierEntreprise`, S3 `verifierCompatibilite`	R2 (entreprise), R7
4	Contrôle de la candidature	REST S4 `verification-doublon`, compte des actives ; date et places d'après S3	R3 à R6
5	Passerelle « Conditions satisfaites ? »	–	–
5	Enregistrement de la candidature	REST S4 `POST /candidatures`	R5, R6 (garanties)
5	Enregistrement du rejet, Notification du rejet	REST S4 `POST /soumissions/{id}/rejet`, S5 notification	–
6	Confirmation du dépôt, Transmission à l'entreprise	REST S5 (deux notifications)	–
7	Décision de l'entreprise (tâche humaine)	contrat {`decision`, `commentaire`}	R9
8	Notification du résultat / du refus	REST S5	–
9	Réservation de la place	SOAP S3 `decrementerPlaces`	R10
9	Enregistrement de l'acceptation / du refus	REST S4 `PATCH /candidatures/{id}/statut`	R9
9	Création du dossier de stage	REST S6 `POST /stages`	R10
9	Mise à jour du statut de l'étudiant	SOAP S1 `changerStatutEtudiant`	R10
–	Demande de complément	REST S5, puis retour à la décision	R11
–	Relance de l'entreprise (minuteur de 7 jours)	REST S5	–

Chaque tâche inscrit son étape dans le journal de S4 (`POST /soumissions/{id}/etapes`) : le portail suit ainsi le cas Bonita en direct.

7.4 Passerelles de décision

Les conditions sont des expressions Groovy sur la variable `dossier`. Chaque passerelle a un flux par défaut, emprunté si aucune condition n'est vraie : le processus ne peut jamais rester bloqué sur une passerelle.

Passerelle	Flux	Condition
Étudiant éligible ?	oui	`dossier.etudiantEligible == true`
	non (défaut)	vers la passerelle de décision, qui rejettera (R1)
Offre existante et active ?	oui	`SoaProcessus.offreValable(dossier)`
	non (défaut)	vers la passerelle de décision (R2)
Conditions satisfaites ?	oui	`SoaProcessus.conditionsSatisfaites(dossier)`
	non (défaut)	enregistrement du rejet, notification, fin « Candidature rejetée »
Décision ?	ACCEPTER	`SoaProcessus.decisionEst(dossier, 'ACCEPTER')`
	REFUSER	`SoaProcessus.decisionEst(dossier, 'REFUSER')`
	DEMANDER_COMPLEMENT	`SoaProcessus.decisionEst(dossier, 'DEMANDER_COMPLEMENT')`

Tableau 7.3 – Conditions des passerelles

Les deux passerelles intermédiaires (« Étudiant éligible ? », « Offre existante et active ? ») ne figurent pas dans l'énoncé. Sans elles, un étudiant ou une offre inexistants feraient échouer l'appel suivant (une offre inconnue n'a pas d'entreprise à vérifier) et le cas partirait en erreur technique, alors qu'il s'agit d'un rejet métier ordinaire. Elles court-circuitent les vérifications devenues inutiles et mènent directement à la passerelle de décision de l'étape 5, qui reste le seul point de rejet.

7.5 Tâche humaine et boucle de complément

« Décision de l'entreprise » est une tâche humaine du couloir Entreprise. Son contrat attend une `decision` (ACCEPTER, REFUSER ou DEMANDER_COMPLEMENT, contrainte vérifiée par Bonita) et un `commentaire`. L'entreprise décide depuis le portail du projet : S4 vérifie d'abord la règle R10 (une place est-elle encore disponible ?), puis recherche la tâche du cas, se l'assigne et l'exécute par l'API de Bonita.

La demande de complément (R11) passe par la tâche « Demande de complément », qui notifie l'étudiant, puis revient à la décision. Un minuteur de sept jours, en bordure de la tâche humaine, déclenche « Relance de l'entreprise » si aucune décision n'est prise ; la tâche de décision est ensuite recréée. La figure 7.3 montre un cas en attente : la tâche humaine et son minuteur sont actifs.

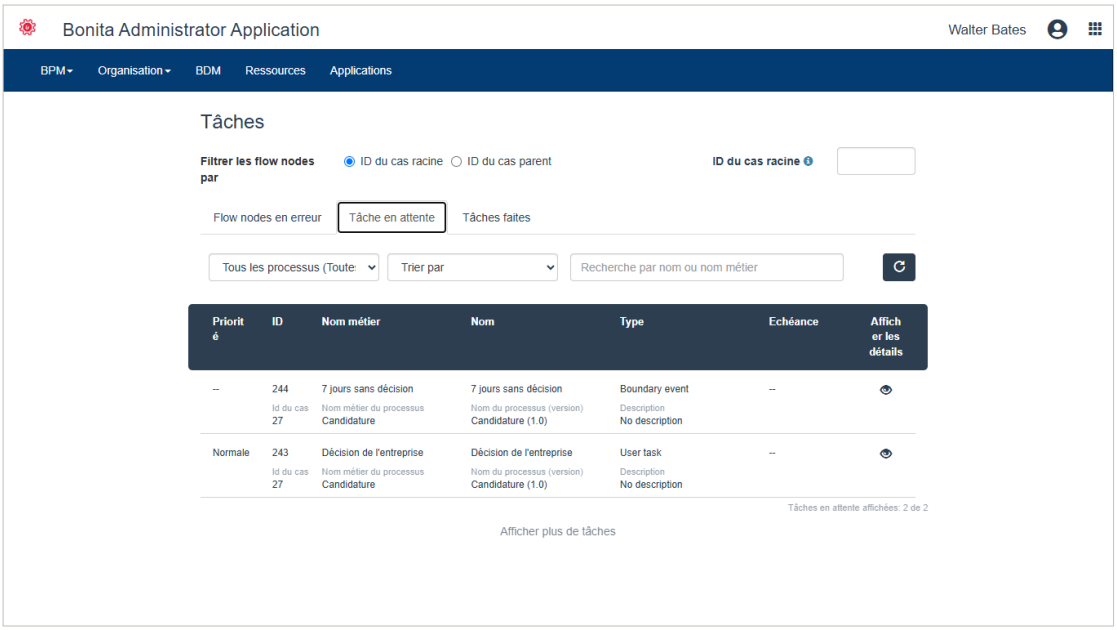


Figure 7.3 – Dans Bonita : la tâche « Décision de l'entreprise » en attente, avec son minuteur de relance

7.6 Gestion des erreurs et des indisponibilités

7.6.1 Dans le modèle

Chaque connecteur est configuré pour lever l'erreur BPMN `SERVICE_INDISPONIBLE` s'il échoue (délai dépassé, réponse 5xx, faute `ServiceIndisponibleFault`). Un unique sous-processus événementiel, « Gestion des erreurs techniques », intercepte cette erreur où qu'elle survienne : « Signaler l'interruption » (S4 passe la soumission en ERREUR, avec la dernière étape réussie), « Alerter le responsable » (courriel `ALERTE_RESPONSABLE`), puis fin « Traitement interrompu ». Un seul sous-processus plutôt qu'un événement de bordure par tâche garde le diagramme lisible et garantit un traitement identique quelle que soit l'étape interrompue.

7.6.2 Dans la version déployée

Sans connecteur, une panne ne peut pas lever d'erreur BPMN : c'est la classe Groovy qui fait le même traitement. Quand un service ne répond pas, la méthode de la tâche signale l'interruption à S4 (soumission en ERREUR), envoie l'alerte au responsable, puis échoue : la tâche apparaît « en erreur » dans Bonita (figure 7.4), où un administrateur peut la rejouer une fois le service rétabli. Côté portail, l'étudiant voit « Traitement interrompu » ; aucune candidature à moitié enregistrée ne subsiste.

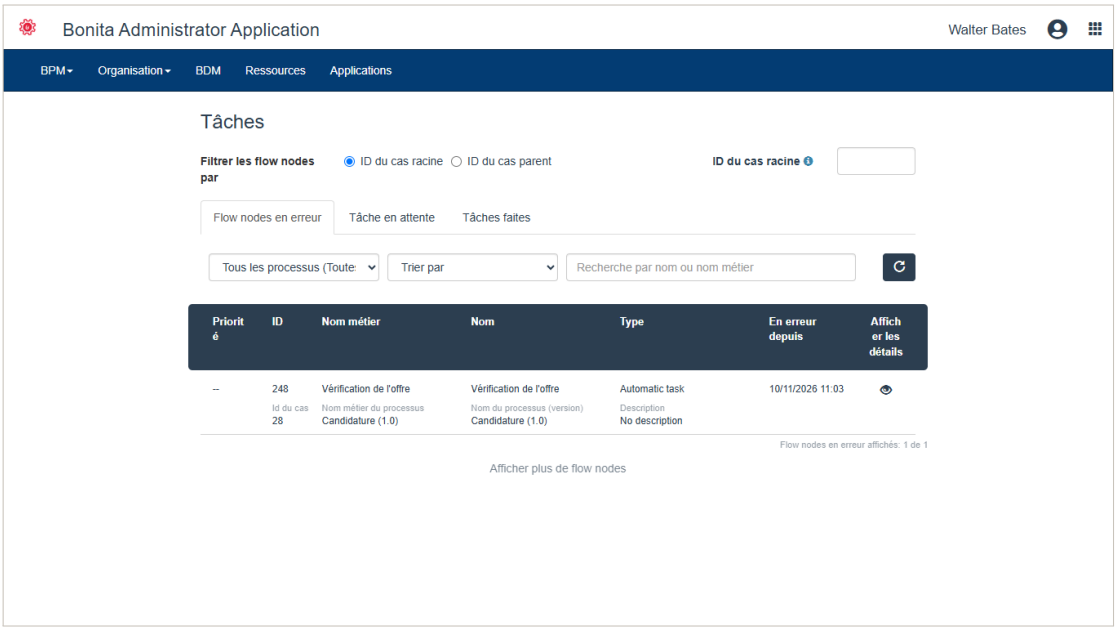


Figure 7.4 – Dans Bonita : la tâche « Vérification de l'offre » en erreur pendant une panne de S3

7.7 Connecteurs du modèle

7.7.1 Connecteurs SOAP

Dans le modèle, les tâches qui appellent S1, S2 ou S3 utilisent le connecteur Web Service (SOAP 1.1) de Bonita. L'enveloppe est produite par la classe Groovy (par exemple `SoaProcessus.enveloppeVerifierEtudiant(dossier)`) et contient l'en-tête WS-Security du compte `bonita` ainsi que l'identifiant de corrélation du cas. La sortie `responseDocumentBody` est lue par la méthode `apres…` correspondante, qui met à jour `dossier` et inscrit l'étape dans le journal de S4.

7.7.2 Connecteurs REST

Les tâches qui appellent S4, S5 ou S6 utilisent le connecteur REST, avec les en-têtes `X-API-Key` (clé du client `bonita`), `X-Correlation-Id` et `Content-Type: application/json`. Les URL et les corps sont, eux aussi, fournis par la classe Groovy. Pour les deux changements de statut (`PATCH`), un connecteur qui ne propose pas ce verbe peut envoyer `POST` avec l'en-tête `X-HTTP-Method-Override: PATCH`, accepté par les services.

7.7.3 Mode secours

En mode secours, la version déployée, chaque tâche est réduite à une seule opération Groovy qui fait l'appel elle-même (par exemple `SoaProcessus.verifierEtudiant(dossier)`), avec les mêmes enveloppes SOAP, en-têtes de sécurité et corrélation que les connecteurs. Le comportement métier est identique.

7.8 Exécution et vérification

Le processus est vérifié à trois niveaux :

- le fichier BPMN est relu sans avertissement par bpmn-moddle, l'analyseur de bpmn-js ;
- une simulation (`tests/bonita/SimulationBonita.groovy`) répond à l'API REST de Bonita et déroule le graphe tâche par tâche avec la classe livrée, avec connecteurs comme en mode secours ;
- surtout, le test de bout en bout a été joué face au **vrai Bonita Community 2025.2**, processus déployé : les 59 vérifications des scénarios et les 6 de la panne de S3 ont réussi, chaque candidature passant par un cas Bonita.

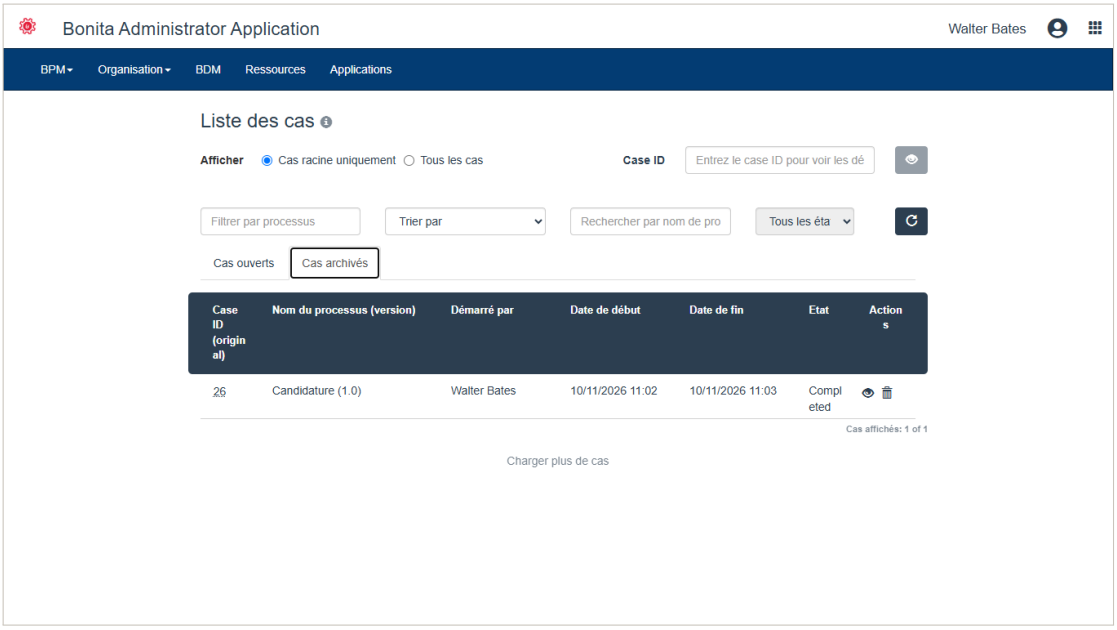


Figure 7.5 – Dans Bonita : le cas d'une candidature acceptée, terminé et archivé

Les captures Bonita de ce chapitre sont produites par `scripts\captures-bonita.ps1`, qui redéploie le processus, joue trois cas réels (acceptation, attente de décision, panne de S3) et capture l'application d'administration de Bonita.

CHAPITRE 8

Sécurité et gestion des erreurs

8.1 Sécurité des échanges SOAP

Chaque appel à S1, S2 ou S3 doit porter un en-tête WS-Security *UsernameToken* avec un mot de passe de type *PasswordText* (listing 8.1). Un gestionnaire JAX-WS commun (`GestionnaireSecuriteServeur`) le vérifie avant que la requête n'atteigne le service ; seule la lecture du WSDL reste publique.

Listing 8.1 – En-tête WS-Security attendu (compte du processus Bonita)

```
<wsse:Security xmlns:wsse="http://docs.oasis-open.org/wss/2004/01/
    ↪ oasis-200401-wss-wssecurity-secext-1.0.xsd">
  <wsse:UsernameToken>
    <wsse:Username>bonita</wsse:Username>
    <wsse:Password>Bonita#Ws2026</wsse:Password>
  </wsse:UsernameToken>
</wsse:Security>
```

Quatre comptes de service existent, un par client : `portail`, `bonita`, `services` (appels entre services) et `soapui` (tests). Le serveur ne connaît que leurs hachages BCrypt, lus dans la configuration (variable d'environnement `SOA_WS_COMPTES` en production) ; chaque appel accepté est journalisé avec le nom du compte, ce qui permet de savoir qui a fait quoi.

Situation	`faultcode`
En-tête de sécurité absent	`wsse:InvalidSecurity`
`Username` ou `Password` manquant	`wsse:InvalidSecurity`
Mot de passe de type *digest*	`wsse:UnsupportedSecurityToken`
Compte inconnu ou mot de passe faux	`wsse:FailedAuthentication`

Tableau 8.1 – Refus d'authentification SOAP

Le mot de passe en clair dans l'en-tête n'est acceptable que sur un canal chiffré : c'est l'objet de la section 8.5.

8.2 Sécurité des API REST

Chaque client des services REST possède sa clé, transmise dans l'en-tête `X-API-Key` : portail, processus Bonita, appels entre services et tests Postman. Les services ne stockent que l'empreinte SHA-256 des clés. Une clé absente ou inconnue donne 401. Certaines opérations sont en outre réservées à certains clients : seul le processus (ou le mode direct) peut enregistrer une candidature ou un rejet, par exemple ; un autre client reçoit 403. Retirer l'accès d'un client revient à supprimer sa clé de la configuration, sans toucher aux autres.

8.3 Sécurité du portail

— **Mots de passe** hachés par BCrypt ; la comparaison prend le même temps qu'un compte existe ou non.
— **Sessions** : un jeton aléatoire de 256 bits est remis à la connexion et envoyé ensuite dans `Authorization: Bearer`. La base n'en garde que l'empreinte SHA-256 ; la session expire au bout de huit heures et la déconnexion la supprime.
— **Rôles** vérifiés côté serveur pour chaque ressource (ETUDIANT, ENTREPRISE, RESPONSABLE) : 401 sans session valide, 403 hors de son espace. Une ressource qui appartient à un autre utilisateur (la candidature d'un autre étudiant, le dossier d'une autre entreprise) répond 404, pour ne pas révéler son existence.
— **Masquage** : quand une entreprise consulte un candidat, le portail masque son courriel et son téléphone (`s***@inpt.ac.ma`, `06******12`) avant l'envoi. Le test de bout en bout vérifie que le JSON reçu par l'entreprise ne contient aucune coordonnée complète.
— **Interface** : tout texte venant des données est inséré comme texte (`textContent`), jamais comme HTML ; les courriels affichés dans l'interface le sont dans un cadre isolé (`iframe sandbox`) sans aucun droit.

8.4 Données et configuration

— **Moindre privilège en base** : chaque service a son compte MySQL, limité à la lecture et à l'écriture des données de son schéma. Un test de socle vérifie qu'un service ne peut ni lire le schéma d'un autre, ni modifier la structure du sien.
— **Requêtes paramétrées** partout (`PreparedStatement`) : aucune concaténation de données dans le SQL.
— **Aucun secret dans le code** : adresses, mots de passe, hachages et clés sont dans la configuration et se remplacent par variables d'environnement. Les valeurs du dépôt sont des valeurs de démonstration.
— **Aucune donnée réelle** : personnes, entreprises et numéros sont fictifs, et les courriels restent dans Mailpit.

8.5 Activer TLS en production

La démonstration fonctionne en HTTP sur une seule machine, sur l'interface locale uniquement (127.0.0.1) : des certificats auto-signés auraient perturbé SoapUI, Postman, Bonita et le navigateur sans rien protéger de plus sur un poste isolé. En production, nous recommandons :

1. un **reverse proxy** (Nginx ou un API gateway) devant le portail, avec un certificat d'une autorité reconnue, TLS 1.2 minimum et l'en-tête HSTS ; le portail ne serait plus joignable directement ;
2. **TLS entre services**, avec une autorité de certification interne, et de préférence l'authentification mutuelle (mTLS), qui authentifie aussi le service appelant ;
3. conserver WS-Security et les clés d'API par-dessus TLS : TLS protège le transport, les jetons identifient le client applicatif ;
4. placer les secrets dans un coffre (variables d'environnement injectées au déploie-

ment) et les renouveler régulièrement.

Côté code, seules les adresses changent (`https://…`) : elles sont toutes dans la configuration.

8.6 Gestion des erreurs

8.6.1 Principes

— **Erreurs prévues, réponses prévues** : faute SOAP typée ou corps JSON uniforme, avec un code stable et un message en français. Aucune trace technique n'est renvoyée au client ; elle est écrite dans le journal du service avec l'identifiant de corrélation.
— **Validation à l'entrée** : tout champ invalide est signalé, avec la liste complète des problèmes (pas seulement le premier).
— **Codes justes** : 400 pour une requête illisible, 422 pour des données invalides, 409 pour une règle de gestion, 503 pour une indisponibilité, 500 seulement pour l'inattendu.

8.6.2 Indisponibilité d'un service

Une panne est traitée à chaque niveau.

Niveau	Comportement
Appel sortant	Délais de connexion (2 s) et de lecture (5 à 8 s) ; l'échec devient une erreur « service indisponible » nommant le service (`SERVICE_OFFRES_INDISPONIBLE`).
Base de données	Pool de connexions avec délai de 3 s ; l'échec devient 503 (REST) ou `ServiceIndisponibleFault` (SOAP) ; `GET /health` répond 503.
Processus BPMN	Sous-processus d'erreur : soumission en ERREUR, responsable alerté par courriel (chapitre 7).
Mode direct	Même comportement : S4 marque la soumission en ERREUR, alerte le responsable et répond 503.
Après une acceptation	Si S6 ou S1 ne répondent pas, l'étape échoue mais la décision est conservée ; le responsable relance le suivi une fois le service rétabli (création du stage idempotente).
Portail	Les pages qui composent plusieurs services restent affichées : si S3 est arrêté, la liste des candidatures s'affiche avec « détails momentanément indisponibles » au lieu d'échouer. Le tableau de bord du responsable indique l'état des six services.

Tableau 8.2 – Traitement d'une indisponibilité

Ces comportements sont vérifiés par de vraies pannes : la suite SoapUI arrête S3, S1 puis MySQL et contrôle les fautes renvoyées ; le test de bout en bout arrête S3 pendant un dépôt et vérifie la soumission en erreur, la réponse 503, l'alerte reçue par le responsable et l'état affiché par le portail (chapitre 9).

8.6.3 Traçabilité

Chaque ligne de journal porte l'identifiant de corrélation de la requête (listing 3.1). Les historiques métier complètent les journaux techniques : historique des statuts d'une candi-

dature (date, acteur, commentaire), journal des étapes du processus, historique des opérations d'un stage, et historique d'envoi de chaque courriel avec son éventuelle erreur SMTP.

CHAPITRE 9

Scénarios de test et résultats

9.1 Stratégie

Les tests sont organisés en quatre niveaux, du plus rapide au plus complet. Chacun se lance par un script et produit un rapport ; les chiffres de ce chapitre sont lus dans ces rapports par `rapport/outils/resultats_tests.py`, qui refuse de produire un tableau si une source manque ou contient un échec.

Niveau	Ce qui est vérifié	Commande
Unitaire et composant	Règles et services sans base (DAO en mémoire), échanges SOAP et REST sur serveur embarqué, conformité aux contrats.	`build.ps1`
Intégration	DAO sur la vraie base MySQL : contraintes, index R5, verrou R6, retrait de place R10.	`build.ps1 clean verify`
Service	Chaque opération SOAP (SoapUI) et REST (Postman), y compris sécurité, erreurs et vraies pannes.	`tests-soapui.ps1`, `tests-postman.ps1`
Bout en bout	Tous les scénarios joués par le portail, courriels vérifiés dans Mailpit, panne de S3, interface dans un navigateur.	`test-bout-en-bout.ps1`, `captures-ui.ps1`

Tableau 9.1 – Niveaux de test

Les tests de service et de bout en bout modifient les données : ils sont joués sur une base fraîche (`reset-db.ps1`). Les données de démonstration fournissent un étudiant différent pour chaque scénario, pour que l'ordre des scénarios n'influe pas sur leur résultat.

9.2 Scénarios fonctionnels

Le tableau 9.2 présente les scénarios principaux, leur résultat attendu et le résultat obtenu lors de la dernière exécution du test de bout en bout (orchestration bonita, processus déployé dans le vrai Bonita 2025.2). La colonne « Obtenu » indique le nombre de vérifications du test qui couvrent le scénario ; toutes ont réussi.

Tableau 9.2 : Scénarios, résultats attendus et obtenus

Scénario	Attendu	Obtenu
Parcours nominal (Salma, offre 1)	Candidature enregistrée, entreprise notifiée, acceptation, stage créé, étudiante EN_STAGE	Conforme (7 vérif.)
R1 : étudiant suspendu ou en stage	Rejet R1, courriel de rejet	Conforme (4 vérif.)
R2 : offre en brouillon, entreprise inactive	Rejet R2, courriel de rejet	Conforme (4 vérif.)
R3 : date limite dépassée	Rejet R3, courriel de rejet	Conforme (2 vérif.)
R4 : plus aucune place	Rejet R4, courriel de rejet	Conforme (2 vérif.)
R5 : candidature en double	Rejet R5, courriel de rejet	Conforme (2 vérif.)
R6 : cinq candidatures actives	Rejet R6, courriel de rejet	Conforme (2 vérif.)
R7 : niveau ou filière incompatible	Rejet R7, courriel de rejet	Conforme (4 vérif.)
R8 : annulation	Refusée (409) pendant l'évaluation, acceptée (204) si déposée	Conforme (3 vérif.)
R11 : demande de complément	Candidature EN_COURS_EVALUATION, étudiant notifié, nouvelle décision possible	Conforme (4 vérif.)
Masquage des coordonnées	L'entreprise ne reçoit ni courriel ni téléphone complets	Conforme (3 vérif.)
Contrôle d'accès	401 sans jeton, 403 hors de son espace, 404 sur le dossier d'un autre	Conforme (6 vérif.)
Rappels de date limite	Courriel aux étudiants actifs compatibles	Conforme (2 vérif.)
Panne de S3	Soumission en ERREUR, réponse 503, responsable alerté	Conforme (6 vérif.)

Le même test passe à l'identique dans les quatre configurations : mode direct, processus simulé avec connecteurs, processus simulé en mode secours, et vrai Bonita avec le processus déployé : 65 vérifications à chaque fois, panne de S3 comprise. Chaque scénario vérifie aussi le courriel reçu dans Mailpit (destinataire, sujet, motif du rejet).

9.3 Tests JUnit

Module	Unitaires	Sur MySQL	Échecs
Socle commun (configuration, sécurité, erreurs)	33	–	0
S1 Étudiants (SOAP)	50	12	0
S2 Entreprises (SOAP)	24	5	0
S3 Offres (SOAP)	56	12	0
S4 Candidatures (REST)	76	7	0
S5 Notifications (REST)	26	3	0
S6 Suivi des stages (REST)	36	5	0
Portail	11	–	0
Total	**312**	**44**	**0**

Tableau 9.3 – Résultats JUnit par module (356 tests)

Au-delà des règles de gestion, ces tests couvrent la validation des entrées, les transitions de statut, la sécurité (jetons WS-Security, clés d'API, sessions et rôles du portail), le masquage, le format des erreurs, la propagation de la corrélation et la conformité des routes REST aux contrats OpenAPI. Les tests du mode bonita utilisent un faux serveur Bonita qui répond à l'API REST, pour vérifier l'instanciation, l'exécution de la tâche de décision et la reprise de session.

9.4 Tests SoapUI

Le projet SoapUI (`tests/soapui/Stages-SOA-soapui-project.xml`) contient six suites. Les trois premières testent chaque opération des trois services : cas nominaux, conformité de la réponse au WSDL, fautes attendues, données invalides, sécurité (sans en-tête, compte inconnu, mauvais mot de passe). Les trois suivantes provoquent de vraies pannes : le script arrête S3, puis S1, puis la base MySQL, vérifie les fautes `ServiceIndisponibleFault` et les réponses dégradées, puis redémarre ce qu'il a arrêté.

Suite	Cas de test	Assertions	Échecs
S1 Étudiants	10	176	0
S2 Entreprises	10	125	0
S3 Offres	12	211	0
Panne : S1 arrêté	1	6	0
Panne : S3 arrêté	1	10	0
Panne : MySQL arrêté	3	18	0
Total	**37**	**546**	**0**

Tableau 9.4 – Résultats SoapUI (546 assertions)

CAPTURE À FAIRE

`rapport/figures/soapui-suite-s1.png`

Voir `docs/CAPTURES_A_FAIRE.md`

Figure 9.1 – SoapUI : suite « S1 Étudiants » exécutée

CAPTURE À FAIRE

`rapport/figures/soapui-requete.png`

Voir `docs/CAPTURES_A_FAIRE.md`

Figure 9.2 – SoapUI : requête `verifierEtudiant` avec en-tête WS-Security

9.5 Tests Postman et Newman

La collection Postman (`tests/postman/Stages-SOA.postman_collection.json`, avec l'environnement `Stages-SOA-local`) compte dix dossiers joués dans l'ordre : sécurité des API, parcours nominal, un rejet par règle R1 à R7, R5 et R6 à l'enregistrement, R8 et R9, R10 (dernière place), R11, validation et erreurs de S4, notifications, suivi des stages. Elle est exécutée en ligne de commande par Newman, qui produit un rapport HTML.

> **Résultat de la dernière exécution**
>
> 87 requêtes, 226 assertions, 0 échec.

CAPTURE À FAIRE

`rapport/figures/postman-runner.png`

Voir `docs/CAPTURES_A_FAIRE.md`

Figure 9.3 – Postman : exécution de la collection

9.6 Interface dans un navigateur

Le script `captures-ui.ps1` pilote Microsoft Edge avec Playwright : il joue une candidature jusqu'à son acceptation par l'interface, visite les vingt-huit écrans des trois espaces et échoue à la moindre erreur dans la console du navigateur, réponse 5xx ou fichier manquant. Les captures du chapitre suivant en sont issues. Ce test a révélé deux défauts corrigés avant la livraison : le message de déconnexion qui disparaissait, et un double

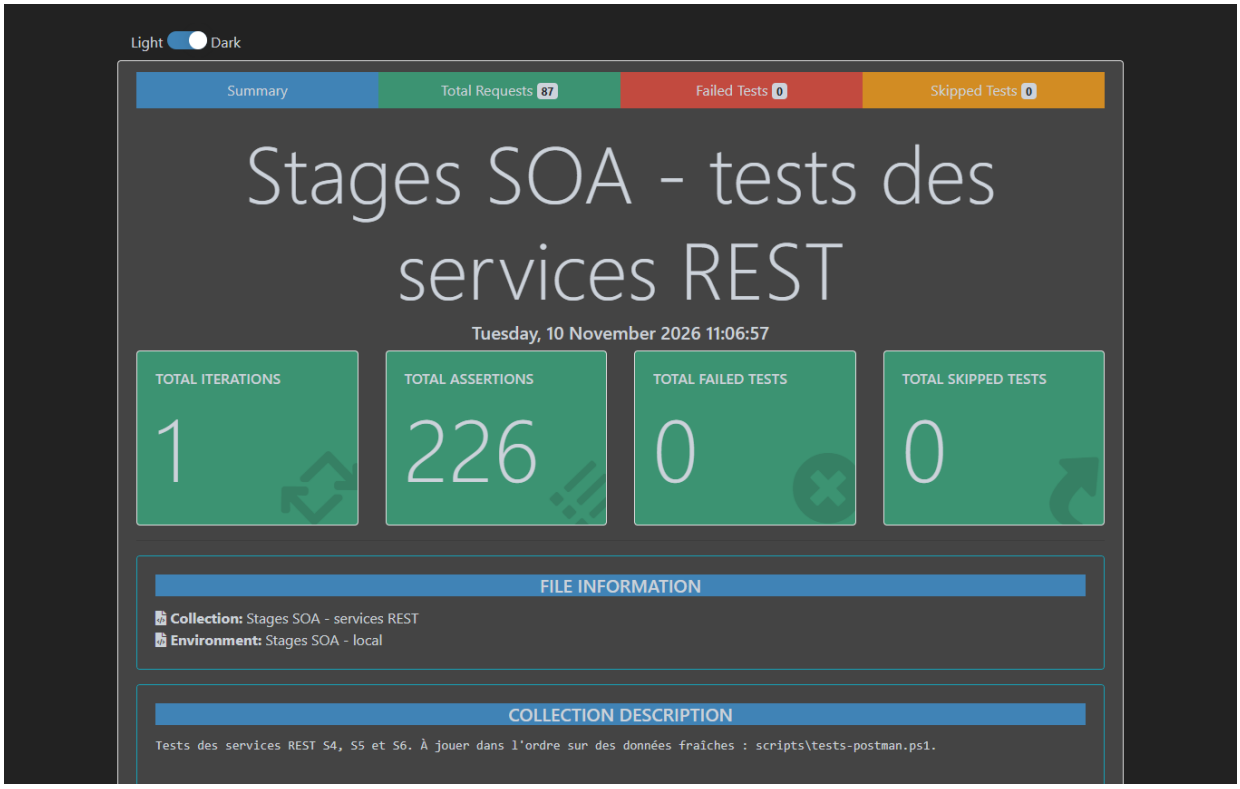


Figure 9.4 – Rapport HTML de Newman

affichage à la connexion.

9.7 Reproduire les résultats

Listing 9.1 – Rejouer l'ensemble des tests (PowerShell, à la racine du projet)

```
.\scripts\reset-db.ps1
.\scripts\build.ps1 clean verify           # JUnit, dont tests sur MySQL
.\scripts\start-all.ps1 -Orchestration direct
.\scripts\verifier-socle.ps1               # socle et donnees de demonstration
.\scripts\tests-soapui.ps1                 # SoapUI, pannes comprises
.\scripts\tests-postman.ps1                # Postman (Newman), base reinitialisee
.\scripts\test-bout-en-bout.ps1            # scenarios par le portail, Mailpit, panne
.\scripts\captures-ui.ps1                  # interface dans Edge
python rapport\outils\resultats_tests.py   # reporte les chiffres dans le rapport
```

CHAPITRE 10

Interface graphique

10.1 Principes

L'interface, proposée en bonus par l'énoncé, est servie par le portail (`http://localhost:8090`). Elle est écrite en HTML, CSS et JavaScript sans framework ni étape de construction : une page unique, des modules JavaScript par espace, un routage par l'adresse (`#/espace/page/identifiant`). Toutes les données passent par l'API du portail, qui applique les rôles et le masquage ; le navigateur n'appelle jamais les services directement.

La charte reste sobre et institutionnelle : le bleu de l'INPT comme seule couleur d'accent, un fond papier, des filets fins, Source Serif 4 pour les titres et Public Sans pour le texte, toutes deux servies par le portail (aucune ressource externe : l'interface fonctionne hors ligne). Les couleurs d'état sont réservées aux statuts et toujours accompagnées d'un libellé en français.

Les captures de ce chapitre ont été produites automatiquement par le parcours Playwright décrit au chapitre précédent, sur les données de démonstration.

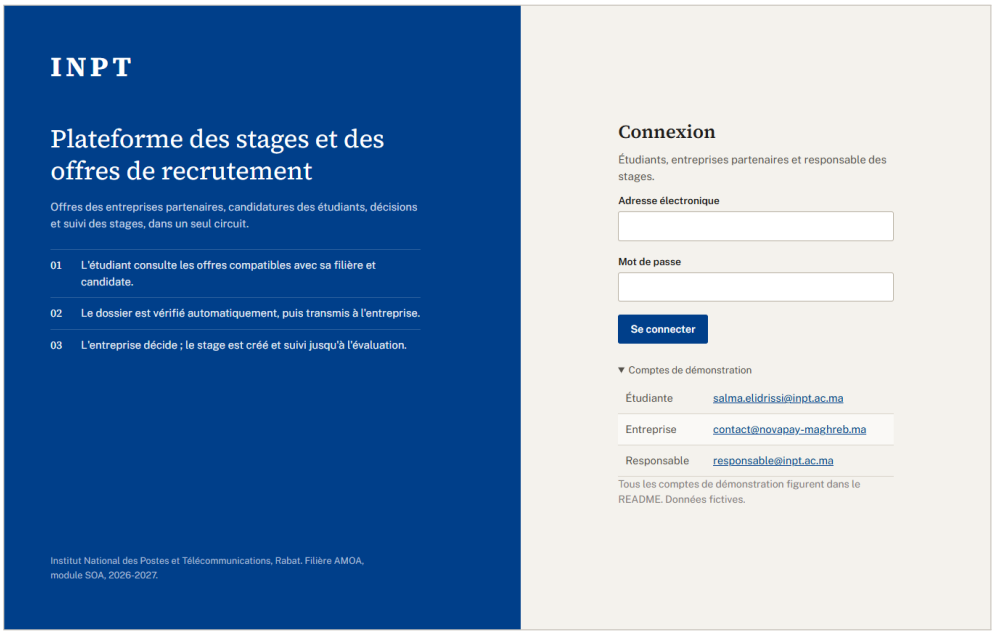


Figure 10.1 – Connexion, avec les comptes de démonstration

10.2 Espace étudiant

L'étudiant consulte les offres ouvertes (date limite non dépassée, au moins une place), filtrables par texte et par filière. Pour chaque offre, l'interface indique si son profil est compatible (règle R7) et le score indicatif de compétences. Il dépose sa candidature avec une lettre de motivation et obtient aussitôt le résultat du contrôle : enregistrée, ou non recevable avec la règle et le motif.

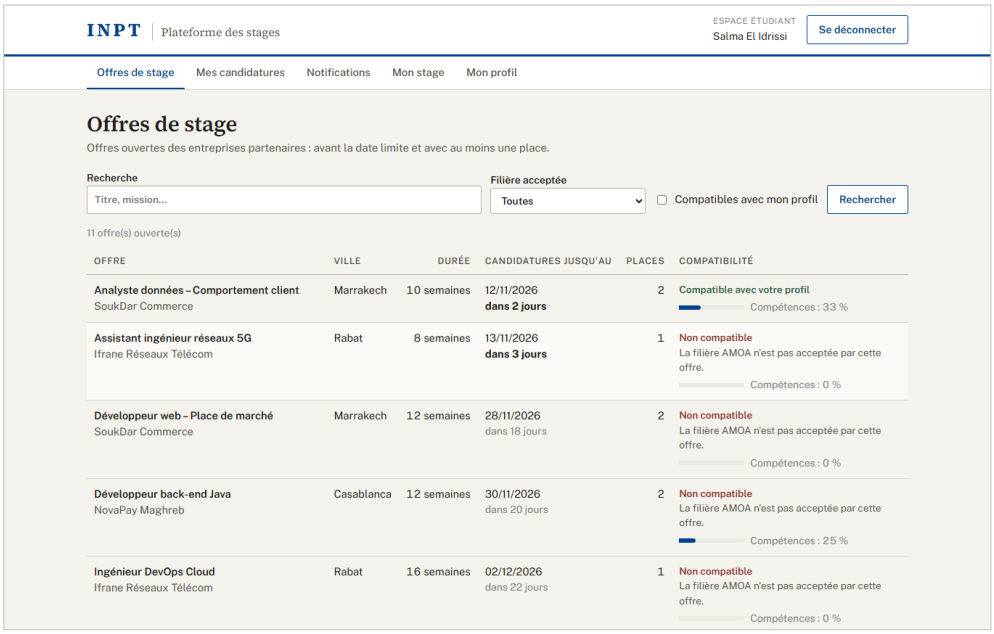


Figure 10.2 – Offres de stage et compatibilité avec le profil

Le dossier d'une candidature réunit le diagramme du processus, coloré à partir du journal des étapes, la liste des étapes (son équivalent textuel), l'historique des statuts et la lettre. Tant que l'entreprise n'a pas ouvert l'évaluation, l'étudiant peut annuler (R8). Ses courriels sont consultables dans l'espace, tels qu'ils ont été envoyés.

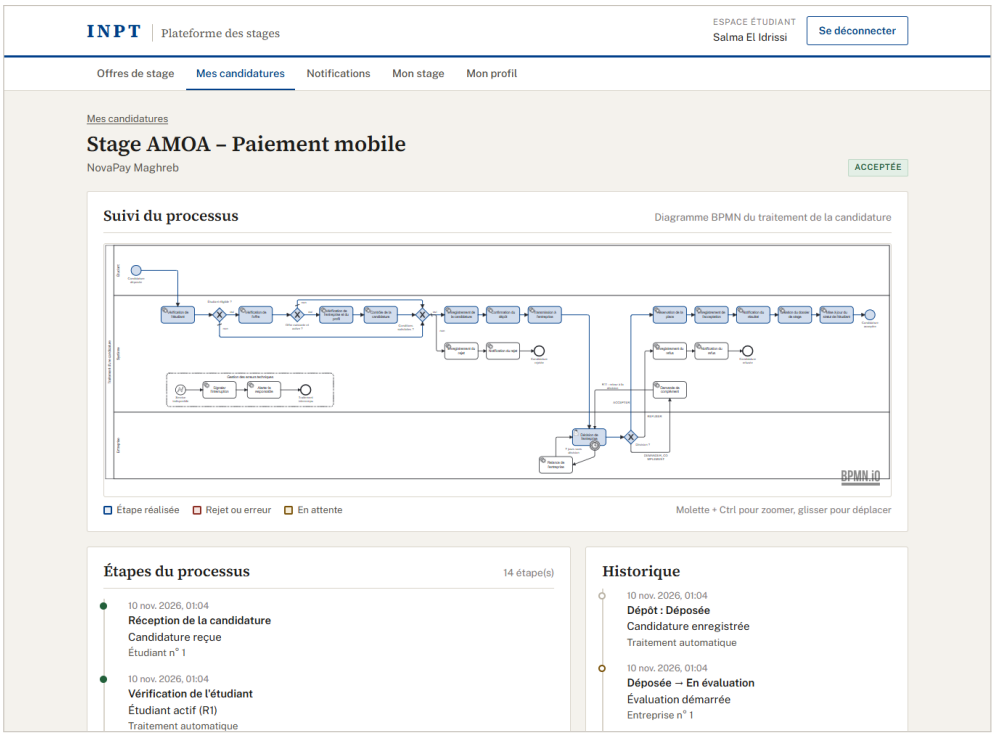


Figure 10.3 – Dossier d'une candidature acceptée : processus BPMN en direct et historique

10.3 Espace entreprise

L'entreprise voit les candidatures reçues pour ses offres, avec le profil du candidat et ses coordonnées masquées. Elle ouvre l'évaluation, puis choisit d'accepter, de refuser ou de demander un complément ; un commentaire est obligatoire pour un complément, et toute décision définitive est confirmée dans une boîte de dialogue. Elle gère aussi ses offres (création, modification selon les transitions permises, clôture) et son profil.

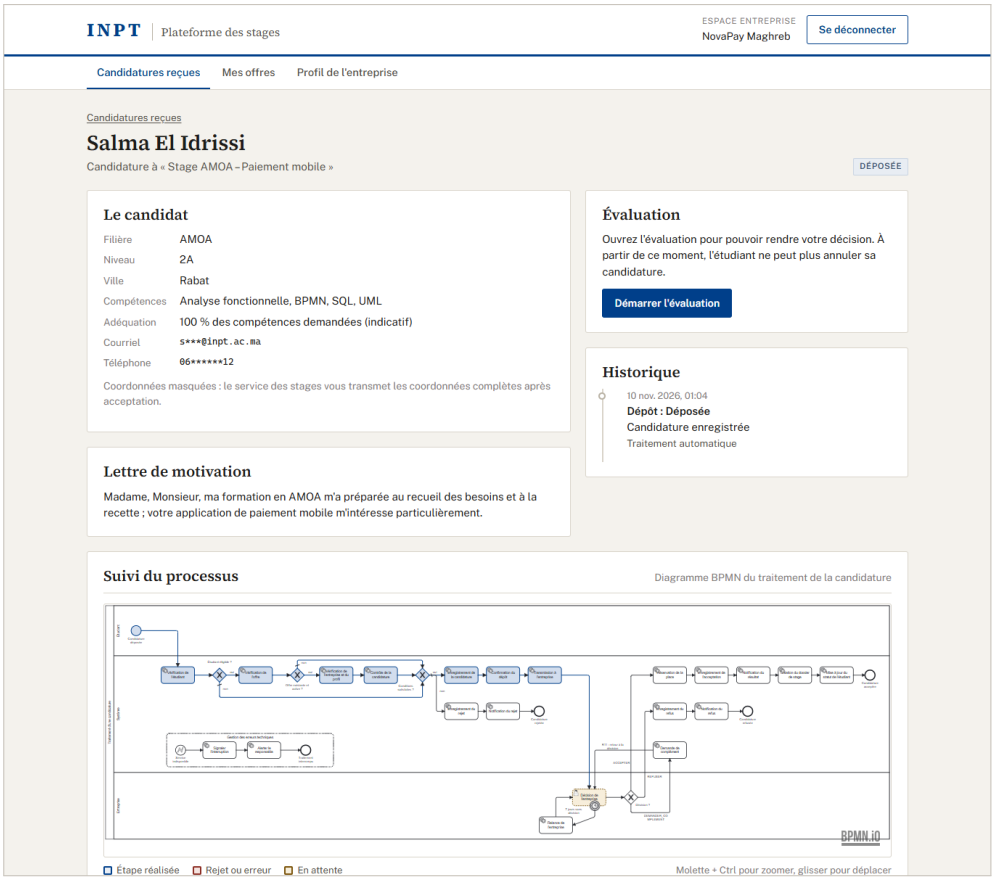


Figure 10.4 – Candidature reçue : profil masqué et ouverture de l'évaluation

10.4 Espace responsable

Le responsable des stages dispose d'un tableau de bord : offres actives, candidatures et décisions en attente, taux d'acceptation, stages en cours, répartition par statut, état des six services et déclenchement manuel des rappels de date limite. Il consulte toutes les candidatures avec les coordonnées complètes, peut relancer le suivi d'une acceptation interrompue par une panne, gère les étudiants et les entreprises, suit les stages (statut, comptes rendus, évaluations) et recherche dans les courriels envoyés.

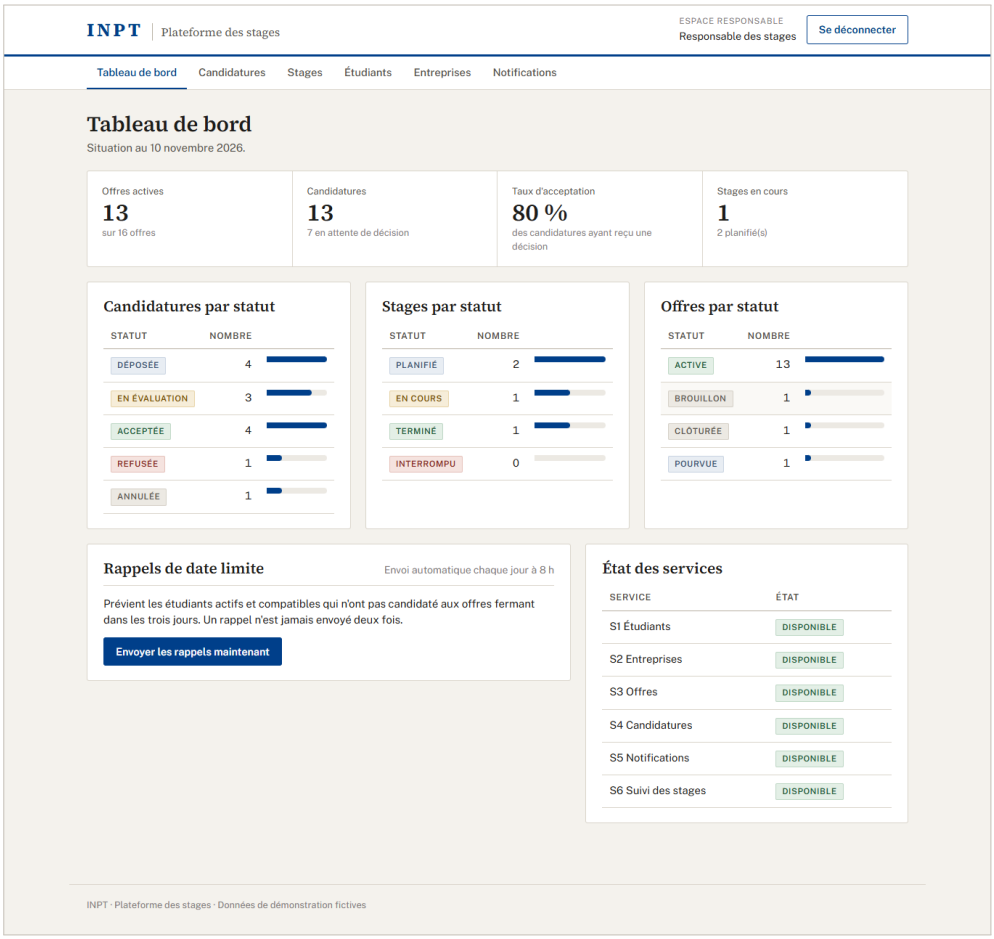


Figure 10.5 – Tableau de bord du responsable des stages

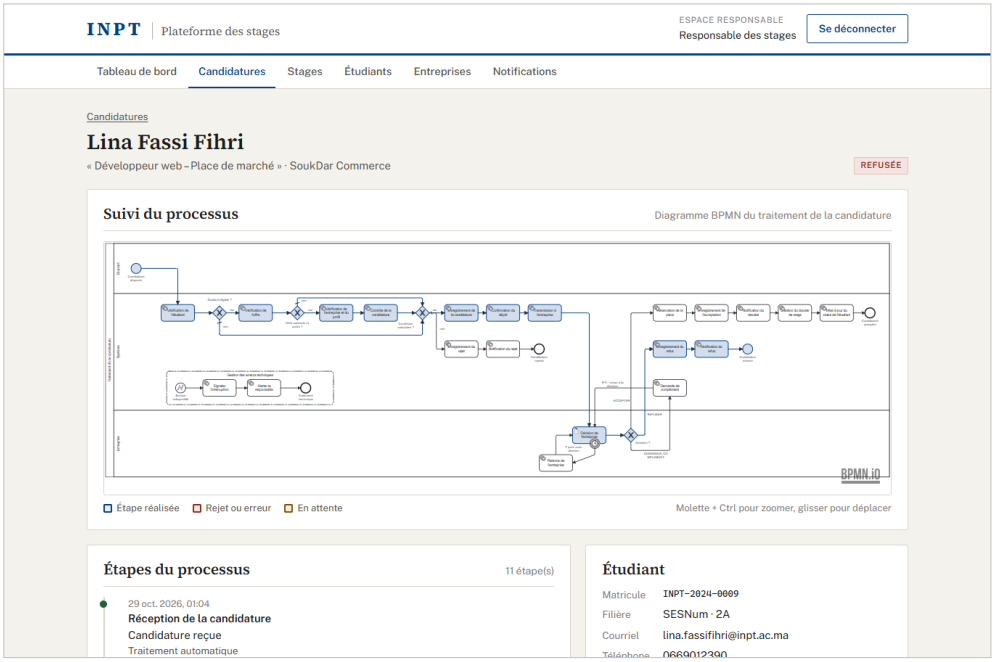


Figure 10.6 – Candidature refusée : branche du refus colorée sur le diagramme

10.5 Vue BPMN en direct

Le diagramme affiché dans les dossiers est le fichier `processus-candidature.bpmn` du processus Bonita, dessiné par la bibliothèque bpmn-js. À l'ouverture d'un dossier, l'interface lit le journal des étapes de la candidature et ajoute un marqueur sur chaque nœud concerné : bleu pour une étape réalisée, rouge pour un rejet ou une erreur, ocre pour l'étape en attente (la décision de l'entreprise). Les flux entre deux étapes réalisées sont également colorés. La branche suivie après la décision (acceptation, refus ou complément) est déduite du détail de l'étape de décision.

Comme les modes direct et bonita écrivent le même journal, la vue est la même quel que soit l'orchestrateur ; en mode bonita, elle suit le cas Bonita au fur et à mesure de son exécution.

CHAPITRE 11

Déploiement et guide d'exécution

11.1 Prérequis

Outil	Version	Usage
JDK	17	Compilation et exécution des services (les scripts vérifient la version).
Maven	3.9	Construction du projet multi-module.
Docker Desktop	avec WSL 2	MySQL 8.4, Mailpit et Adminer.
Bonita Studio Community	2025.2	Exécution du processus BPMN (mode bonita).
SoapUI Open Source	5.10	Tests SOAP.
Node.js	LTS	Newman, test de bout en bout, Playwright (installés automatiquement par les scripts).
Microsoft Edge	–	Parcours de l'interface par Playwright.

Tableau 11.1 – Logiciels nécessaires

Tous les scripts sont en PowerShell et se lancent depuis la racine du projet. Si Windows refuse leur exécution : `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`.

11.2 Démarrage

Listing 11.1 – Démarrage complet

```
.\scripts\start-all.ps1 -Rebuild -Orchestration direct   # construit, demarre Docker puis les
    ↪ 7 services
.\scripts\check-health.ps1                               # etat de chaque service
```

`start-all.ps1` démarre les conteneurs (MySQL, Mailpit, Adminer), attend que MySQL soit prêt, puis lance chaque service dans son propre processus Java, journal dans `logs/`. Il affiche ensuite l'état de chacun (`GET /health`) et le mode d'orchestration. Au premier démarrage, MySQL crée les sept schémas, les comptes de service et les données de démonstration (`db/init`).

Composant	Adresse
Interface (portail)	`http://localhost:8090`
Services SOAP (WSDL)	`http://localhost:8081/ws/etudiants?wsdl`, `:8082/ws/entreprises?wsdl`, `:8083/ws/offres?wsdl`
Services REST (Swagger UI)	`http://localhost:8084/docs/`, `:8085/docs/`, `:8086/docs/`
Boîte de courriel Mailpit	`http://localhost:8025`
Administration MySQL (Adminer)	`http://localhost:8088` (serveur `mysql`)
Bonita	`http://localhost:8080/bonita`

Tableau 11.2 – Adresses utiles

Rôle	Identifiant	Mot de passe
Responsable des stages	`responsable@inpt.ac.ma`	`Responsable@2026`
Étudiante	`salma.elidrissi@inpt.ac.ma`	`Etudiant@2026`
Entreprise	`contact@novapay-maghreb.ma`	`Entreprise@2026`

Tableau 11.3 – Comptes de démonstration (tous les étudiants et entreprises ont un compte)

11.3 Mode d'orchestration

— **bonita** (par défaut) : le processus doit être déployé dans Bonita. Bonita Studio ouvert (il démarre le serveur Bonita), une commande suffit : `.\scripts\deployer-bonita.ps1`. La version à connecteurs se construit à la main dans le Studio en suivant `docs/bonita/GUIDE_BONITA.md`. S4 vérifie au démarrage que Bonita répond ; sinon, `start-all` l'indique et les dépôts répondent 503.
— **direct** : aucune dépendance à Bonita ; S4 exécute les mêmes étapes. C'est le mode des tests automatiques.

Le mode se choisit au démarrage : `start-all.ps1 -Orchestration bonita` ou `direct`.

11.4 Configuration

Chaque paramètre a une valeur par défaut adaptée à la démonstration et peut être remplacé, sans recompiler, par une variable d'environnement (nom en majuscules, points remplacés par des soulignés : `soa.db.port` devient `SOA_DB_PORT`).

Variable	Rôle
`ORCHESTRATION_MODE`	`bonita` ou `direct`
`SOA_DB_HOTE`, `SOA_DB_PORT`, `SOA_DB_PASSWORD`	Base de données
`SOA_WS_URL_*`, `SOA_REST_URL_*`	Adresse de chaque service appelé
`SOA_WS_COMPTES`, `SOA_API_CLES`	Hachages des comptes WS-Security et des clés d'API acceptés
`SOA_WS_DELAI_LECTURE_MS`, `SOA_REST_DELAI_LECTURE_MS`	Délais d'attente des appels
`BONITA_URL`, `BONITA_UTILISATEUR`, `BONITA_MOTDEPASSE`	Accès à l'API de Bonita

Tableau 11.4 – Principales variables de configuration

11.5 Opérations courantes

Listing 11.2 – Scripts d'exploitation

```
.\scripts\stop-all.ps1          # arrete les services et Docker (donnees conservees)
.\scripts\reset-db.ps1          # remet les bases et Mailpit a l'etat initial
.\scripts\verifier-socle.ps1    # 68 verifications : bases, droits, donnees de demonstration
.\scripts\build.ps1             # compile et lance les tests unitaires
.\rapport\compiler.ps1          # compile ce rapport (XeLaTeX)
```

Avant une démonstration, `reset-db.ps1` remet les données dans leur état initial : chaque scénario dispose alors de son étudiant (`docs/SCENARIOS_DEMO.md`).

11.6 Dépannage

Symptôme	Cause probable et remède
`start-all` signale Docker absent	Démarrer Docker Desktop, attendre qu'il soit prêt, relancer.
Un service reste à 503 sur `/health`	Sa base ne répond pas : `docker ps`, puis `reset-db.ps1` si besoin ; voir `logs/<service>.log`.
Dépôt de candidature en 503 en mode bonita	Bonita n'est pas démarré ou le processus n'est pas déployé ; ou démarrer en mode direct.
Un rejet R1 inattendu pendant la démonstration	Les données ont été modifiées par un test : `reset-db.ps1`.
Port déjà utilisé	Une ancienne instance tourne : `stop-all.ps1 -ServicesSeuls`.

Tableau 11.5 – Problèmes fréquents

CHAPITRE 12

Limites, améliorations et conclusion

12.1 Limites connues

— **Pas de HTTPS en local.** Les mots de passe WS-Security et les clés d'API circulent en clair sur l'interface locale de la machine. C'est un choix assumé pour la démonstration ; la section 8.5 décrit la mise en production.
— **Une seule instance par service**, sans répartition de charge. Les services sont sans état (hors base), ce qui permettrait de les dupliquer, mais rien n'est prévu pour les découvrir automatiquement.
— **Cohérence entre services.** L'acceptation modifie trois services sans transaction distribuée. L'ordre des étapes, l'idempotence de la création du stage et la reprise par le responsable évitent toute incohérence durable, mais la reprise reste manuelle.
— **Bonita.** La version déployée utilise le mode secours (sans connecteur) : une panne y met la tâche en erreur au lieu de déclencher le sous-processus d'erreur du modèle, et le diagramme n'est pas éditable dans le Studio (il est construit par programme ; la version à connecteurs se construit à la main avec le guide). Le compte technique qui exécute les décisions masque, dans Bonita, l'auteur réel de la décision (connu de S4). Supprimer un cas à l'annulation efface son historique dans Bonita (il reste dans S4).
— **Interface.** Pas de version mobile travaillée ; pas de notification en direct (les écrans se mettent à jour à la navigation) ; le compte de connexion d'un étudiant créé par le responsable n'est pas créé automatiquement.
— **Infrastructure.** Seules la base, Mailpit et Adminer sont conteneurisés ; les services Java se lancent par des scripts PowerShell, donc sous Windows.

12.2 Améliorations possibles

Amélioration	Apport
API gateway	Point d'entrée unique devant les services REST (Kong, Spring Cloud Gateway…) : authentification, quotas, journalisation et TLS centralisés ; le portail ne s'occuperait plus que de l'interface.
ESB	Médiation entre SOAP et REST, transformation des messages et routage (Apache Camel, WSO2) : les services n'auraient plus à connaître le protocole des autres.
Registre de services	Découverte dynamique des adresses (Consul, Eureka) au lieu d'adresses fixes en configuration ; prérequis pour plusieurs instances.
Conteneurisation complète	Une image Docker par service et un `docker compose up` unique, puis Kubernetes : démarrage identique sur toute machine, redémarrage automatique, mise à l'échelle.
Messagerie asynchrone	Notifications publiées dans une file (RabbitMQ, Kafka) : un serveur de courriel lent ou arrêté ne ralentirait plus le traitement d'une candidature.
Saga	Compensations automatiques pour l'acceptation (rendre la place si la création du stage échoue définitivement), au lieu de la reprise manuelle.
Identité fédérée	OAuth 2 et OpenID Connect (Keycloak) pour les utilisateurs et les clients techniques, avec jetons signés à durée de vie courte.
Observabilité	Traces distribuées (OpenTelemetry) prolongeant l'identifiant de corrélation, métriques et tableaux de bord.
Intégration continue	Construction, tests et rapport à chaque modification du dépôt.

Tableau 12.1 – Pistes d'amélioration

12.3 Conclusion

Ce projet met en œuvre, sur un cas métier concret, les principes d'une architecture orientée services : des services autonomes qui possèdent leurs données, exposent un contrat explicite (WSDL ou OpenAPI) et ne communiquent que par ces contrats ; un processus BPMN qui orchestre ces services pour traiter une candidature de bout en bout ; une sécurité adaptée à chaque type d'échange.

Au-delà des fonctionnalités demandées, nous avons cherché à rendre le système vérifiable : chaque règle de gestion porte un identifiant suivi du code jusqu'aux courriels, chaque contrat est confronté au code par des tests, chaque panne est provoquée réellement pour en vérifier le traitement, et une candidature peut être suivie d'un service à l'autre grâce à un identifiant de corrélation. Les 356 tests JUnit, les 546 assertions SoapUI, les 226 assertions Postman et le test de bout en bout ont tous réussi lors de la dernière exécution.

Le projet nous a surtout appris que la difficulté d'une architecture orientée services n'est pas d'écrire chaque service, mais de traiter ce qui se passe entre eux : un service qui ne répond pas, deux demandes simultanées, une étape qui réussit à moitié. Les choix les plus utiles (une soumission distincte de la candidature, un journal d'étapes commun aux deux modes d'orchestration, des opérations idempotentes, un sous-processus d'erreur unique) sont nés de ces questions.